

Архитектура 12 STOREEZ

Внутренняя техническая документация

01.08.2026

Оглавление

Архитектура 12 STOREEZ	6
Состав	6
С чего начать	6
Ключевые цифры	7
Как поддерживать	7
Общая картина	8
Что это за система	8
Два стека в одном репозитории	8
Точки входа	9
Конфигурация	9
Схема запроса	11
Как система работает асинхронно	11
Что читать дальше	12
Бэкенд	13
Два стека: <code>modules/</code> и <code>src/</code>	13
DI-контейнер	19
События	20
Общие каталоги вне модулей	21
Консольные команды	21
Статический анализ	22
Бизнес-домены	23
Каталог и товары	23
Корзина и чекаут	25
Заказы	27
Платежи	29
Доставка	31
Пользователи и аутентификация	31
Контент и CMS	32
Рекомендации и персонализация	32
Параметры и фича-флаги	33
Другие домены (кратко)	34
База данных	35
Общие сведения	35
Соглашения об именовании	35
Ключевые таблицы по доменам	36

Связи ключевых сущностей	39
Служебные и легаси-таблицы	42
Миграции	42
Способы доступа к данным	44
Поиск	45
Redis	45
API	47
Три генерации API	47
Маршрутизация	47
Аутентификация.....	48
Эндпоинты.....	49
Формирование ответа.....	57
Валидация	58
Контракты и документация.....	58
Вебхуки и входящие интеграции	59
gRPC	60
Экспорт фидов	61
Фронтенд	62
Сборка	62
Связка PHP и Vue	65
Модули.....	67
ui-kit.....	68
Состояние.....	69
Аналитика и внешние сервисы.....	70
Параллельный редизайн.....	71
Легаси	73
Структура frontend/src/	73
Инфраструктура	74
CI/CD	74
Топология в Kubernetes.....	76
Асинхронная обработка.....	80
Кэширование.....	81
Наблюдаемость	82
Конфигурация и секреты	83
Внешние интеграции	86
Сводная таблица.....	86
Внутренние микросервисы.....	90
Особенности отдельных интеграций	90

Переменные окружения.....	91
Схема потоков данных.....	93
Неочевидные связи и подводные камни	94
Критичное.....	94
Архитектурные ловушки	95
База данных	97
Кэш.....	98
Фича-флаги	99
Фронтенд.....	99
Инфраструктура	101
Идемпотентность и повторные попытки	102
Быстрый чек-лист перед изменением	104

Архитектура 12 STOREEZ

Описание фактического устройства монолита: слои, домены, БД, API, фронтенд, инфраструктура. Документация описывает состояние ветки `master` и опирается на код, а не на планы. Все утверждения прошли независимый аудит против исходного кода (сентябрь 2026).

Единый PDF со всеми разделами и кликабельным оглавлением: <dist/12storeez-architecture.pdf>.
Пересобрать после правок: `python3 docs/architecture/build-pdf.py` (нужны `pandoc`, `weasyprint`, `@mermaid-js/mermaid-cli` — см. комментарий в начале скрипта).

Состав

Документ	О чём
01-overview.md	Общая картина: два стека, точки входа, конфигурация
02-backend.md	<code>modules/</code> и <code>src/</code> , слои, DI-контейнер, консольные команды
03-business-domains.md	Каталог, корзина, заказы, платежи, доставка, пользователи, CMS
04-database.md	Схема, ключевые таблицы и связи, миграции
05-api.md	Три генерации API, аутентификация, эндпоинты, контракты
06-frontend.md	Сборка, связка PHP [?] /Vue, модули, ui-kit, редизайн
07-infrastructure.md	CI/CD, Kubernetes, очереди, кэш, наблюдаемость
08-integrations.md	Внешние системы и переменные окружения
09-pitfalls.md	Неочевидные связи и подводные камни

С чего начать

- **Новый разработчик** — [01-overview.md](#), затем [02-backend.md](#) или [06-frontend.md](#) по своему профилю, и обязательно [09-pitfalls.md](#).
- **Правка бизнес-логики** — [03-business-domains.md](#) + [04-database.md](#).
- **Новый эндпоинт** — [05-api.md](#).
- **Новая интеграция или очередь** — [07-infrastructure.md](#) + [08-integrations.md](#).

Ключевые цифры

Метрика	Значение
Легаси-модули (<code>modules/</code>)	64
Современные модули (<code>src/Modules/</code>)	43
Контроллеры (всего / админских)	388 / 127
ActiveRecord-модели с <code>tableName()</code>	240
Миграции	196 (апрель 2025 — август 2026)
Модули фронтенда / entry points	58 / 95
Консольные команды	94
Строк в DI-контейнере	1516

Как поддерживать

Документация описывает структуру, а не отдельные фичи. При изменениях обновлять нужно, если:

- появился новый модуль в `modules/` или `src/Modules/` ;
- добавлен платёжный провайдер, внешняя интеграция или очередь;
- изменилась схема аутентификации или версионирование API;
- изменился способ сборки фронтенда или связка PHP[?]Vue.

Описание отдельных фич живёт рядом по теме — например, `docs/sale/categories-sale.md` .
Контракты API — в `docs/frontend/openapi.yml` , `docs/mobileApi/openapi.yml` , `docs/ssr/openapi.yml` .

Общая картина

Что это за система

Интернет-магазин 12 STOREEZ. Монолит на PHP 8.2 / Yii2 с фронтендом на Vue 2, работает в Kubernetes. Обслуживает три типа клиентов: сайт, мобильные приложения (iOS/Android) и партнёрские интеграции (1C, RetailCRM, Lamoda, Mercaux).

Технологический стек:

Слой	Технологии
Бэкенд	PHP 8.2, Yii2 2.0.55
Фронтенд	Vue 2, Webpack 5, SCSS
БД	MariaDB/MySQL (префикс таблиц <code>cms_</code>), отдельная БД для логов
Кэш	Redis (два кластера)
Очереди	MySQL-очередь + RabbitMQ
Поиск	Elasticsearch
Внешние вызовы	HTTP REST, SOAP (1C), gRPC (микросервисы)

Два стека в одном репозитории

Главная особенность проекта: **бэкенд состоит из двух параллельных стеков**, живущих на одной БД.

	<code>modules/</code>	<code>src/</code>
Namespace	<code>modules\</code>	<code>Application\</code>
Стиль	Yii2 MVC, «толстые» ActiveRecord	Слои Service / Repository / Dto / Enum
Что содержит	Модели данных, CMS-админка, views, легаси-кроны	Новый чекаут, mobile API, платежи, outbox, интеграции
Объём	64 каталога	43 домен-модуля в <code>src/Modules/</code> + 11 инфраструктурных каталогов рядом

Разделение **не по слоям, а по времени написания**. Практическое правило:

- **данные и админка** — в `modules/`;
- **новые клиентские флоу и интеграции** — в `src/`.

Важная оговорка: многие «современные» пункты в этом описании — это не отдельные каталоги `modules/<имя>/`, а **Yii ID модулей**, зарегистрированные в `config/modules.php` и указывающие на классы из `src/Modules/`. Например `productV2`, `carts`, `mobile-v2` физически лежат только в `src/`, отдельных каталогов `modules/productV2/` и т.п. не существует. Подробнее — [02-backend.md](#), раздел «Дублирование модулей».

Важно понимать, что это не «легаси и новый код», а два работающих стека с взаимными зависимостями. `src/` активно использует модели из `modules/`, а `modules/` использует инфраструктуру из `src/` (Redis, очереди). Подробнее — [09-pitfalls.md](#), раздел про инвертированные зависимости.

Единственная точка связывания — DI-контейнер `di/ContainerRegister.php`, который регистрирует классы обоих стеков. См. [02-backend.md](#).

Точки входа

Файл	Назначение
<code>www/index.php</code>	Веб: <code>vendor/autoload.php</code> → <code>config/config.php</code> (только в <code>\$params</code>) → <code>vendor/yiisoft/yii2/Yii.php</code> → <code>config/web.php</code> → <code>yii\web\Application</code>
<code>yii</code>	Консоль: <code>vendor/autoload.php</code> → <code>Yii.php</code> → <code>config/console.php</code> → <code>yii\console\Application</code>
<code>Yii.php</code>	Файл фреймворка Yii2 (не проектный класс), подключается обоими входами до чтения конфигов

Каталог `themes/basic/` — отдельный слой PHP-представлений (180+ файлов: layouts, views модулей, асеты), через который проходит связка с фронтендом. Подробнее — [06-frontend.md](#).

Конфигурация

Конфиги собираются каскадом: `common.php` — общая база, `web.php` и `console.php` её расширяют.

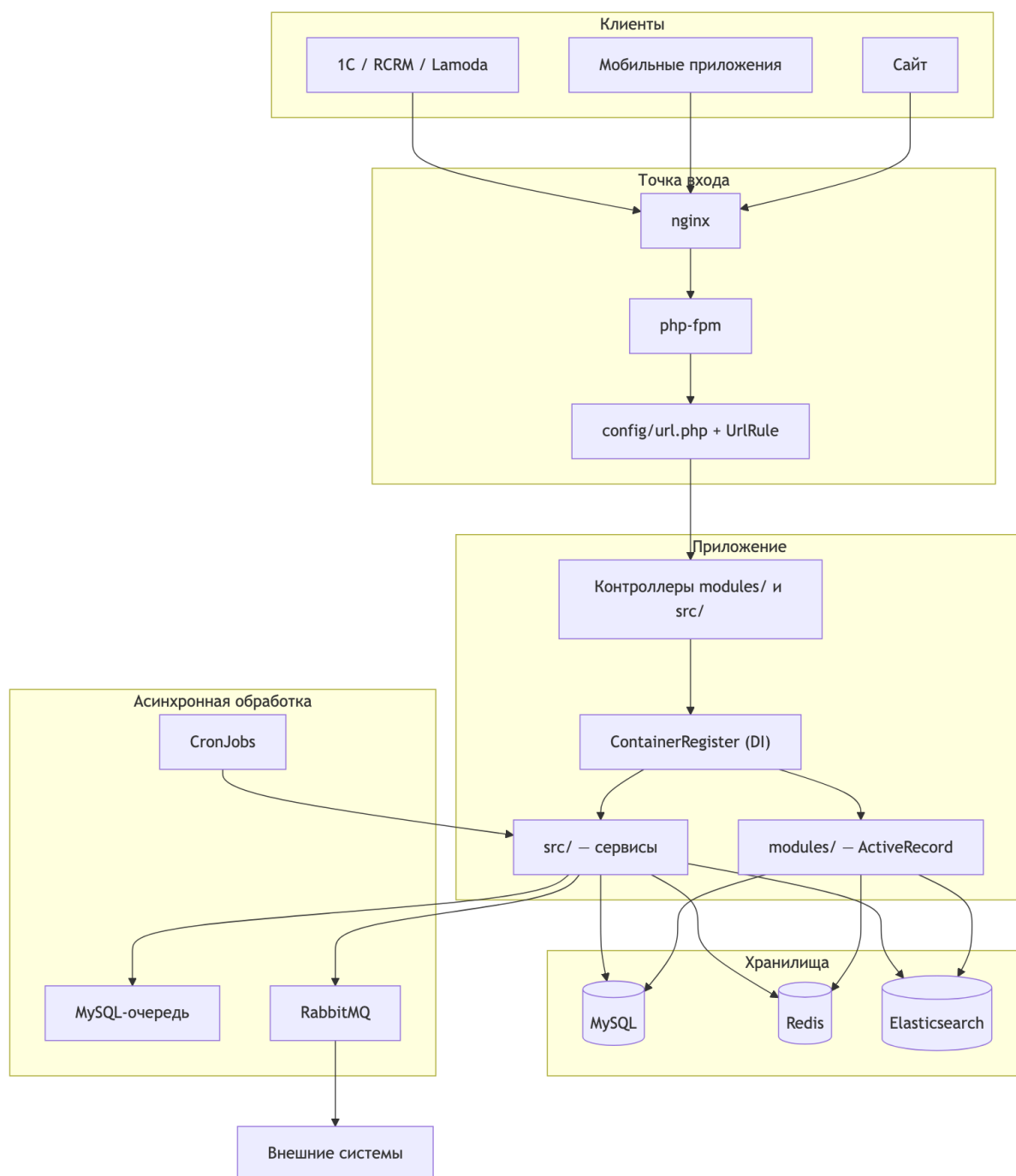
Файл	Содержимое
<code>config/config.php</code>	Чтение переменных окружения в массив <code>\$params</code>
<code>config/common.php</code>	Компоненты для web и консоли: <code>db</code> , отдельное подключение <code>log_db</code> , <code>queue</code> , <code>mindbox</code> , <code>retailCrm</code> , службы доставки, <code>atol</code> , <code>sentry</code> , RBAC
<code>config/web.php</code>	Web-специфика: <code>request</code> , <code>user</code> , кэш, <code>i18n</code> , <code>urlManager</code> , обработчики <code>beforeRequest</code> / <code>afterRequest</code> , OAuth
<code>config/console.php</code>	94 консольные команды через <code>controllerMap</code>
<code>config/modules.php</code>	Регистрация Yii-модулей из обоих стеков
<code>config/url.php</code>	Правила маршрутизации (336 строк) + кастомные <code>UrlRule</code>
<code>config/log.php</code>	Цели логирования: БД, stdout/stderr в JSON, файл
<code>config/events.php</code>	Обработчики событий приложения
<code>config/sphinx.php</code>	Не используется в рантайме , см. 09-pitfalls.md

Переменные окружения в проде приходят из Vault через секрет `web-monolith-secret-envs`, локально — из `.env` (в git не хранится, пример — `.env.example`).

Redis сконфигурирован как два независимых логических кластера: основной (`REDIS_CLUSTER_NODES` / `REDIS_HOST`) и отдельный для нового чекаута (`REDIS_NEW_CLUSTER_NODES`), переключаемый по URI запроса. Подробнее — [07-infrastructure.md](#).

Отдельный слой вне `modules/` и `src/` — каталог `components/` (Yii-компоненты уровня приложения: `SqlQueue`, `QueueController`, `rabbitmq/`) и сгенерированные gRPC-клиенты в `modules/grpc/` (`Mindbox`, `Feedbacks`, `Orders`). Подробнее — [02-backend.md](#).

Схема запроса



Как система работает асинхронно

Значительная часть логики выполняется вне веб-запроса:

- **MySQL-очередь** (таблица `default_queue`, без префикса `TABLE_PREFIX`) — 22 именованные очереди, воркеры `yii queue/listen <name>`.
- **RabbitMQ** — 45 констант очередей (40 уникальных имён, часть констант — маппинги на уже объявленные), отдельные деплойменты-консьюмеры.

- **Outbox/Inbox** — обмен с 1C, RetailCRM, Mindbox и платёжными провайдерами идёт через таблицы-outbox в MySQL, откуда кроны и воркеры отправляют сообщения в Rabbit.
- **CronJobs** — 109 расписаний в Helm: фиды, переиндексация, обработка outbox, уведомления.

Ключевой вывод: **рантайм-поведение прода описано в `.deploy/helm/values.yaml`**, а не в коде приложения. Консольная команда может существовать без воркера, и наоборот. См. [07-infrastructure.md](#).

Что читать дальше

- Устройство бэкенда и DI — [02-backend.md](#)
- Бизнес-домены и их флоу — [03-business-domains.md](#)
- Подводные камни, которые стоит знать до первой правки — [09-pitfalls.md](#)

Бэкенд

Два стека: `modules/` и `src/`

`modules/` — Yii2 MVC

64 каталога, namespace `modules\<имя>\`. Ориентировочная структура модуля (реально присутствует не везде — см. ниже):

```
modules/<имя>/
├── controllers/      # web + *BackendController (админка)
├── models/          # ActiveRecord, часто с бизнес-логикой
├── service/ | services/
├── repository/ | repositories/
├── views/
├── console/         # консольные контроллеры
├── components/
└── Module.php
```

Это не универсальный шаблон: одновременно `controllers/` + `models/` + `views/` + `Module.php` есть только у 40 из 64 каталогов. У 10 каталогов `Module.php` нет вовсе (`common`, `feed`, `grpc`, `gii`, `mainpage`, `mindbox`, `mobile`, `shopping`, `smsRateLimiter`, `userActivities`) — это общие библиотеки или каталоги, подключаемые без регистрации как Yii-модуль. `console/` встречается в 17/64, `components/` — в 10/64. Там, где `Module.php` есть, он обычно наследует `modules\cms\Module`; явные исключения, наследующие `\yii\base\Module` напрямую: `modules/cms/Module.php` (сам базовый класс), `modules/onelink/Module.php`, `modules/rates/Module.php`, `modules/diginetica/Module.php`.

Здесь живут модели данных и вся CMS-админка. Админские контроллеры называются `*BackendController` и доступны по маршруту `cms/<модуль>/<контроллер>-backend/<действие>` — их 127 из 388 контроллеров.

Крупнейшие классы проекта — именно здесь, и они содержат бизнес-логику:

Класс	Файл	≈ строк
Cart	modules/cart/models/ Cart.php	3329
Product	modules/product/models/ Product/Product.php	3028
Order	modules/orders/models/ Order.php	2976
Category	modules/catalog/models/ Category.php	2177
ProductService	modules/product/service/ ProductService.php	1951
ProductAggregate	modules/product/models/ Product/ ProductAggregate.php	1665
OrderPosition	modules/orders/models/ OrderPosition.php	1483
User	modules/users/models/ User.php	1445
OrderService	modules/orders/services/ OrderService.php	1396

Список не исчерпывающий — среди крупных каталогов `modules/`, не разобранных подробно в этом документе: `cms/` (138 файлов, базовый Yii-фреймворк админки), `product/` (289 файлов), `catalog/` (95), `common/` (66, легаси-слой без своего `Module.php`), `grpc/` (72, сгенерированный `protobuf`-код).

`src/` — слоистая структура

Namespace `Application\`, 43 домен-модуля в `src/Modules/` плюс 11 инфраструктурных каталогов рядом (всего 12 каталогов верхнего уровня в `src/`, включая сам `Modules/`).

Инфраструктурные каталоги `src/`:

Каталог	Назначение
Auth/	JWT-аутентификация как Yii-модуль
Cache/	Redis: подключения, контракты, конфигураторы кластеров
Cart/	Тонкий мост — единственный файл <code>Service/CartService.php</code> , не путать с полноценным доменом <code>src/Modules/Cart/</code> (188 файлов)
Common/	Bootstrap событий, базовое подключение к БД, RabbitMQ, S3, rate limiter
Framework/	LogJsonTarget
Http/	Контроллеры API, валидация запросов, сервис формирования ответа
Integration/	Внешние системы: платежи Yandex, капча, email-уведомления, микросервисы
Monitoring/	Sentry
Queue/	Имена очередей (<code>SqlQueueName</code> , <code>RabbitMqQueueName</code>), хелперы DI
Search/	Elasticsearch: клиент, индексатор, построитель запросов
User/	REST API пользователя + подмодуль уведомлений

Крупнейшие домен-модули `src/Modules/` по числу файлов:

Модуль	Файлов	Ключевые слои
Order	197	32 сервиса, 24 репозитория, 26 обработчиков AfterSaveEvent , 26 DTO
Cart	188	75 сервисов (фабрики под режимы корзины), 25 фильтров (12 платёжных + 13 доставки/прочих), 29 DTO
Mobile	105	38 контроллеров mobile API v1, 16 валидаторов
Payment	80	outbox-сервисы, клиенты провайдеров, консольные обработчики
Receipt	70	фискальные чеки, ATOL, receipt MS
Delivery	69	построение вариантов доставки, гео
Common	69	параметры-флаги, middleware, локализация
Product	59	пара к легаси modules/ product/ : рекомендации, ранний доступ, остатки
Mindbox	44	REST- и gRPC-обёртки над CRM
MobileV2	40	контроллеры второй версии mobile API (Yii ID mobile-v2)
Stock	21	буферы и консольные команды синхронизации остатков (ESB)

Список неполный — в src/Modules/ также есть Rcrm , Images , Notifications , Reservation , Geo , Giftcard , Api2 , Cms , YandexTracker , MobileInfo , Landing , Poll , Vacancy , Blog , Navigation , ActionPresent , Analytics и другие домены меньшего размера.

Соглашения по именованию в src/

Единообразия нет — в разных модулях встречаются оба варианта:

- Service/ и Services/
- Controller/ и Controllers/
- Helper/ и Helpers/
- Exception/ и Exceptions/
- Model/ и Models/

Суффиксы устойчивы: *Dto , *Enum , *Dictionary , *Service , *Repository , *Extractor , *Transformer , интерфейсы — в Interfaces/ .

Дублирование модулей

Важно: таблица ниже — это про **Yii ID модулей** в `config/modules.php`, а не про пары одноимённых каталогов. У большинства «современных» ID нет каталога `modules/<id>/` вообще — их класс модуля зарегистрирован напрямую на `Application\Modules\...\Module` из `src/`. Каталог `modules/<имя>/` существует **только у легаси-стороны** каждой пары.

Легаси: Yii ID → каталог	Современный: Yii ID → класс	Как разделены
<code>product</code> → <code>modules/product/</code>	<code>productV2</code> → <code>Application\Modules\Product\Module</code> (только <code>src/</code>)	CRUD каталога легаси; рекомендации и API — в <code>src/</code>
<code>cart</code> → <code>modules/cart/</code>	<code>carts</code> → <code>Application\Modules\Cart\Module</code> (только <code>src/</code>)	Оба ID зарегистрированы с одним и тем же <code>'controllers' => ['cart']</code> ; UI старой корзины — легаси, новый чекаут-API — в <code>src/</code>
<code>orders</code> → <code>modules/orders/</code>	(своего Yii ID нет) <code>src/Modules/Order/</code> подключается напрямую, без записи в <code>config/modules.php</code>	Админка и страницы оплаты — легаси; создание заказа, outbox — в <code>src/</code>
<code>payment</code> → <code>modules/payment/</code>	<code>paymentV2</code> → <code>Application\Modules\Payment\Module</code> (только <code>src/</code>)	Админка чеков — легаси
<code>mobilev2</code> → <code>modules/mobilev2/</code>	<code>mobile-v2</code> → <code>Application\Modules\MobileV2\Module</code> (только <code>src/</code> , каталога <code>modules/mobile-v2/</code> нет)	Общий префикс URL / <code>mobile/v2/</code> — см. 09-pitfalls.md
<code>retailCrm</code> → <code>modules/retailCrm/</code>	<code>rcrm</code> → <code>Application\Modules\Rcrm\Module</code> (только <code>src/</code>)	Синхронизация через outbox — в <code>src/</code>
<code>users</code> → <code>modules/users/</code>	<code>user</code> , <code>userApi</code> , <code>userNotifications</code> — три отдельных ID, все на классы из <code>src/</code> (<code>Application\Modules\User\Module</code> , <code>Application\User\Module</code> , <code>Application\User\Notifications\Module</code>); каталогов <code>modules/user/</code> , <code>modules/userApi/</code> нет	Профиль и админка — легаси; импорт, REST-профиль и уведомления — в <code>src/</code>
<code>delivery</code> → <code>modules/delivery/</code>	<code>deliveryV2</code> и <code>return</code> — оба указывают на один и тот же класс <code>Application\Modules\Delivery\Module</code> , различаются только набором <code>controllers</code>	Модель данных — легаси; расчёт вариантов и обработка возвратов — в <code>src/</code>
<code>lookbook</code> → <code>modules/lookbook/</code>	<code>lookbookv2</code> → <code>Application\Modules\Lookbook\Module</code> (только <code>src/</code>)	
<code>marketing</code> → <code>modules/marketing/</code>	<code>marketingV2</code> → <code>Application\Modules\Marketing\Module</code> (только <code>src/</code>)	
<code>subscription</code> → <code>modules/subscription/</code>	<code>subscriptions</code> → <code>Application\Modules\Subscription\Module</code> (только <code>src/</code>)	
<code>wishlist</code> → <code>modules/wishlist/</code>	<code>wishlists</code> → <code>Application\Modules\Wishlist\Module</code> (только <code>src/</code>)	

Легаси: Yii ID → каталог	Современный: Yii ID → класс	Как разделены
<code>feedback</code> → <code>modules/feedback/</code>	<code>feedbackV2</code> → <code>Application\Modules\Footer\Module</code> (только <code>src/</code>)	
<code>api2</code> → <code>modules/api2/</code>	<code>api2_new</code> → <code>Application\Modules\Api2\Module</code> (только <code>src/</code>)	Старые и новые вебхуки

Особые случаи, не укладывающиеся в схему «легаси-каталог + современный ID»:

- **feed** — Yii ID **один**, `Application\Modules\Footer\Module` из `src/`; отдельного legacy-модуля нет. При этом каталог `modules/feed/` существует (35 файлов: старые `extractors/` `services/console` для экспорта) и используется напрямую консольными командами, без прохождения через систему Yii-модулей.
- **mainpage** — аналогично, Yii ID один и указывает на `Application\Modules>Mainpage\Module`. Каталог `modules/mainpage/` при этом существует (6 файлов) — там остались legacy-views, которые современный модуль по-прежнему использует для рендера.

Только в `src/`, без каких-либо legacy-остатков в `modules/`: `auth`, `checkout`, `metadata`, `menu`, `metrics`, `oncs`, `analytics`.

Ещё один пример не столь очевидного дублирования: позиции корзины лежат в отдельном модуле `modules/shopping/` (`ShoppingCartPosition`), а не в `modules/cart/`, хотя сама корзина (`Cart`) — в `cart`. Механизмы `outbox/inbox` и gRPC-клиенты в `modules/grpc/` здесь не расписаны подробно — см. [07-infrastructure.md](#) и [05-api.md](#), раздел «gRPC».

DI-контейнер

`di/ContainerRegister.php` — 1516 строк, реализует `BootstrapInterface`, подключается в `config/common.php`. Содержит **341** вызов `setSingleton()` и импортирует **381** класс из обоих стеков.

Что делает:

- регистрирует конкретные классы как синглтоны;
- связывает интерфейсы с реализациями;
- создаёт фабрики для Redis, gRPC, JWT, капчи, обработчиков Yandex-платежей;
- конфигурирует клиентов по `Yii::$app->params`.

Основные связи интерфейс → реализация:

Интерфейс	Реализация
RedisInterface	ClusterConnection или StandaloneConnection
NewRedisInterface	NewClusterConnection или NewStandaloneConnection
UserMobileSessionRepositoryInterface	CacheUserMobileSessionRepository
CaptchaInterface	ReCaptchaService
LoggerInterface	Application\Modules\Common\Service\Logger
FieldMapperInterface	ElasticCatalogFieldMapperService
DeliveryTkEntityRepositoryInterface	CacheDeliveryTkPvzRepository

Выбор реализации Redis зависит от окружения: если заполнены переменные с узлами кластера — кластерное подключение, иначе одиночное.

Отдельная группа регистраций — мосты к легаси: `OldCartService`, `OldOrderService`, `OldUserService`, `OldPaymentTypeService`, `OldImageService`, `OldDeliveryTkRepository`, `OldOrderPositionRepository`, `OldOrderRepository`. Они регистрируются рядом с новыми сервисами, из-за чего граница между стеками намеренно проницаема.

Дополнительные синглтоны есть и в `config/common.php` в секции `container.singletons` — например, `AuthClientInterface` подменяется на `LocalClient` вне прода.

Практическое следствие: менять `ContainerRegister.php` рискованно — от него зависят оба стека, и файл не разделён на провайдеры по доменам.

События

Два независимых механизма.

1. События приложения — `src/Common/Bootstrap/EventBootstrap.php` регистрирует: `UserAfterLoginEvent`, `UserAfterEditEvent`, события смены пароля, `OrderAfterCreateEvent`, `OrderAfterUpdateEvent`. Каждый наследует `BaseEvent`.

2. Побочные эффекты заказа — 26 классов в `src/Modules/Order/AfterSaveEvent/`, запускаются через `src/Modules/Order/Services/EventRunner.php` после коммита транзакции: отправка в Mindbox, переиндексация товаров, блокировка сертификатов, отмена BNPL-платежей, аналитика.

Отдельно в `config/events.php` — единственный обработчик: обновление кэша меню навигации (`eventHandlers/MenuCacheReloaderHandler.php`).

Общие каталоги вне модулей

Каталог	Файлов	Содержимое
<code>components/</code>	33	Yii-компоненты: <code>SqlQueue</code> , <code>RabbitMQ</code> , <code>MindboxComponent</code> , <code>Mailer</code> , <code>LangRequest</code> , <code>reo (Dadata</code> , <code>Google</code> , <code>Geocode</code>), партнёрские (<code>Admitad</code> , <code>GdeSlon</code>)
<code>helpers/</code>	7	<code>StringHelper</code> , <code>TransliterateHelper</code> , <code>MobileVersionChecker</code> , <code>ColorHelper</code> , <code>DbHelper</code> , <code>ImageHelper</code> , <code>MoneyFormatHelper</code>
<code>coreExtends/</code>	10	<code>ExtendedErrorHandler</code> , HTTP-исключения (<code>Redirect301Exception</code>), расширенное логирование
<code>widgets/</code>	6	Легаси-UI: пагинация, нотификации, подписка
<code>validators/</code>	7	Валидаторы авторизации по SMS, телефона, заявки на возврат
<code>eventHandlers/</code>	1	Обработчик обновления кэша меню
<code>translations/</code>	—	YAML-переводы для Symfony Translator (домены <code>cart/</code> , <code>order/</code> и др.)
<code>themes/basic/</code>	—	PHP-views сайта, точка связки с фронтендом

Консольные команды

94 команды регистрируются в `config/console.php` через `controllerMap`. Namespace `app\commands`, указанный как `controllerNamespace`, физически не существует (нет ни каталога, ни PSR-4 записи в `composer.json`) — команды приходят только из `controllerMap`. Группы:

Группа	Примеры команд
Очереди	<code>queue</code> , <code>slave-queue</code> , <code>rabbit-inbox-transactions</code> , <code>rabbit-outbox-transactions</code> , <code>cron-outbox-transactions</code> , <code>rabbit-bnpl-outbox-transactions</code> , <code>receipt-outbox</code>
Платёжные уведомления	<code>notification-podeli</code> , <code>notification-yandex-split</code> , <code>notification-yandex-pay</code> , <code>notification-yandex-sbp</code> , <code>change-status-podeli</code> , <code>change-status-dolyame</code>
Заказы и интеграции	<code>order</code> , <code>lamoda-order</code> , <code>offline-order</code> , <code>send-order-1c</code> , <code>bug-orders</code> , <code>mindbox-console</code> , <code>rcrm</code> , <code>cron-onec-settings</code>
Каталог	<code>export</code> , <code>feed-export</code> , <code>exchange</code> , <code>product-reindex</code> , <code>product-import</code> , <code>product-attributes</code> , <code>early-access</code> , <code>elastic</code> , <code>stocks</code>
Доставка и гео	<code>delivery</code> , <code>boxberry</code> , <code>accord</code> , <code>geo</code> , <code>ipgeobase-parse</code>
Пользователи	<code>user-import</code> , <code>jwt-auth</code> , <code>check-user-address</code> , <code>DuplicateMerge</code> , <code>sms-log</code>
Чеки	<code>receipt</code> , <code>atol</code> , <code>lifepay</code>
Обслуживание	<code>migrate</code> , <code>sitemap</code> , <code>clear-log</code> , <code>clear-db</code> , <code>clear-outbox</code> , <code>cache</code> , <code>message</code>

Файл `config/cron-web` описывает расписания времён bare-metal и **в Kubernetes не используется** — актуальные расписания в Helm.

Статический анализ

Конфиг	Уровень	Область
<code>phpstan.neon</code>	2	<code>components</code> , <code>config</code> , <code>modules</code> , <code>www</code> и др. — <code>src/</code> не входит , 181 строка <code>excludedPaths</code> (не <code>ignoreErrors</code> — этой секции в файле нет)
<code>phpstan_6lvl_src.neon</code>	6	только <code>src/</code> , подключается отдельной job при изменениях в <code>src/**</code>

Дополнительно: `phpcs.xml` (code style), `phpunit.xml` (тесты в `tests/`).

Обратите внимание на асимметрию: строгая проверка применяется к `src/`, который зависит от слабо проверяемого `modules/`. Последствия описаны в [09-pitfalls.md](#).

Бизнес-домены

Для каждого домена указано, где лежит код, какие перечисления определяют поведение и как распределена логика между стеками.

Каталог и товары

Где код

Область	Путь
Модель товара	<code>modules/product/models/Product/Product.php</code> , <code>ProductAggregate.php</code>
Размеры	<code>modules/product/models/ProductSize.php</code> , <code>ProductSizeAggregate.php</code>
Теги товара	<code>modules/product/service/ProductTagsService.php</code>
Ранний доступ	<code>src/Modules/Product/Service/EarlyAccessConditionsService.php</code> , <code>EarlyAccessAvailabilityService.php</code>
Категории	<code>modules/catalog/models/Category.php</code> , <code>modules/product/repository/ProductCategoryRepository.php</code>
Остатки (легаси)	<code>modules/catalog/models/Stock.php</code> , <code>ItemStock.php</code> , <code>modules/product/service/CachedStocksService.php</code>
Остатки (новые)	<code>src/Modules/Product/Models/ProductStock.php</code> , <code>Repository/ProductStockRepository.php</code> , <code>src/Modules/Stock/</code>
Цены и скидки	<code>modules/processing/service/ProcessingService.php</code> , <code>modules/discounts/</code>
Агрегация каталога	<code>src/Modules/Product/Service/CatalogAggregator.php</code>
Индексация	<code>src/Search/Elastic/Service/ElasticCatalogIndexerService.php</code>

Модель данных товара

Важная особенность: **товар в одном цвете — это отдельная строка в `products`** . Справочник цветов (`dictionary_colors`) есть, но отдельной таблицы «цветовой вариант товара» нет — каждый цвет модели хранится полноценной строкой `products` со ссылкой `color_id` → `dictionary_colors` . Цветовые варианты одной модели связаны общими полями `group_guid` и `super_model_guid` .

Это влияет на аналитику, фиды и подсчёт «числа товаров»: одна модель в пяти цветах — это пять записей каталога.

Теги

Отдельного enum для всех тегов нет, логика собрана в `ProductTagsService`: ранний доступ, предзаказ, sample sale, отсутствие в наличии, только в магазинах, процент скидки, «Скоро» (по `Category::COMING_SOON_ID`), кастомный тег.

Признак «Скоро» хранится полем `products.is_coming_soon` и блокирует покупку, если товар не идёт по флору предзаказа.

Атрибуты товара — отдельная сущность с enum:

```
// src/Modules/Product/Enum/ProductAttributeEnum.php
case SALE = 'sale';
case EARLY_ACCESS = 'early_access';
```

Логика SALE-режима подробно описана в `docs/sale/categories-sale.md`.

Доступность и остатки

`ProductStockRepository` рассчитывает несколько независимых пулов: `qty_site`, `qty_shops`, `qty_cnc`, `qty_offline_shipment` и отдельный `qty_site_early_access` для раннего доступа. Ранний доступ включается флагом `early_access_enabled`.

Фиды

Экспорт запускается только из консоли, результат — XML/CSV файлы.

Получатель	Реализация
Mindbox	<code>modules/feed/extractors/xml/MindboxFeedExtractor.php</code>
RetailCRM	<code>modules/feed/extractors/xml/CrmFeedExtractor.php</code>
Google Merchant, Google Ads	<code>GoogleFeedExtractor</code> , <code>MerchantFeedExtractor</code>
Yandex (Market, Direct, видео)	<code>modules/feed/extractors/</code>
VK, Criteo, Admitad, ретаргетинг	<code>modules/feed/extractors/</code>
1C	<code>OdinAssFeedExtractor</code>
Garderobo, Dresscode, Tbank, Zosper	<code>src/Modules/Feed/Service/</code>

Старые фиды — в `modules/feed/console/ExportNewController.php`, новые — в `src/Modules/Feed/Controller/FeedExportController.php`.

Корзина и чекаут

Режимы корзины

Поведение корзины определяется режимом, под каждый режим есть свой сервис:

```
// src/Modules/Cart/Enum/CartModeEnum.php
MODE_ORDER      = 'order'           // обычный заказ
MODE_PREORDER    = 'preorder'        // предзаказ
MODE_CNC_LIGHT   = 'cnclight'        // click & collect light
MODE_EXPRESS     = 'express_delivery' // экспресс-доставка
MODE_EXPRESS_PICKUP = 'express_pickup' // экспресс-самовывоз
MODE_RESERVATION = 'reservation'     // резерв в магазине
```

Режим определяется `ModeHelper::getMode()`, сервис выбирается через `src/Modules/Cart/Service/CartServiceFactory.php`.

Где код

Область	Путь
Модель корзины (общая)	<code>modules/cart/models/Cart.php</code>
Базовая логика	<code>src/Modules/Cart/Service/AbstractCartService.php</code> (~2600 строк)
Реализации режимов	<code>CartService/CartService.php</code> , <code>ExpressCartService.php</code> , <code>PreorderCartService.php</code> , <code>ShopCartService.php</code> — все в <code>src/Modules/Cart/Service/</code> ; <code>ReservationCartService.php</code> — в другом модуле, <code>src/Modules/Reservation/Service/</code> , импортируется в <code>CartServiceFactory</code>
Web API	<code>src/Modules/Cart/Controllers/CartApiController.php</code> , <code>CartDeliveryController.php</code> , <code>CartPaymentController.php</code>
Mobile API	<code>src/Modules/Mobile/Controllers/Cart*.php</code>
Страница нового чекаута	<code>src/Modules/Checkout/Controllers/CheckoutController.php</code>
Легаси-корзина	<code>modules/cart/controllers/CartController.php</code>
Бонусы	<code>modules/mindbox/services/MindboxBonusService.php</code>
Промокоды	<code>modules/processing/</code> , <code>modules/discounts/models/PromoCode.php</code>
Сертификаты	<code>modules/giftcards/service/GiftcardsService.php</code> , <code>src/Modules/Giftcard/</code>
Выбор оплаты	<code>src/Modules/Cart/Service/CartPaymentServiceFactory.php</code>

Флоу от добавления до чекаута

1. Определение режима — `ModeHelper` → `CartServiceFactory` .
2. Получение или создание корзины — `AbstractCartService::getCart()` , ключ по пользователю или сессии плюс тип корзины.
3. Добавление и изменение позиций — проверка остатков через `ProductStockService` , раннего доступа через `EarlyAccessAvailabilityService` , правила режима.
4. Доставка — `DeliveryVariantsService` и `CartDeliveryValidator` : город, служба, ПВЗ, интервалы.
5. Скидки — промокод через `ProcessingService` , бонусы через `Mindbox` , сертификаты через `GiftcardsService` .

6. Список способов оплаты — `CartPaymentTypeService::getPaymentFilters()` фильтрует по службе доставки, сумме заказа и платформе через 12 классов-фильтров (`src/Modules/Cart/Filter/Payment/`); `CartPaymentServiceFactory` выбирает не фильтры, а конкретный `CartPaymentService*` под режим корзины. Рядом в `src/Modules/Cart/Filter/Delivery/` — фильтры доставки; всего в каталоге `Filter/` 25 файлов, включая абстрактные классы и интерфейсы.

Заказы

Где код

Область	Путь
Модель (современная)	<code>src/Modules/Order/Model/Order.php</code>
Модель (легаси)	<code>modules/orders/models/Order.php</code>
Создание	<code>src/Modules/Order/Services/CreateOrder/</code> — <code>AbstractCreateOrderService</code> , <code>CreateOrderServiceSite</code> , <code>CreateOrderServiceMobile</code> , <code>CreateReservationServiceMobile</code>
Сборка DTO	<code>src/Modules/Order/Builder/</code> <code>CreateOrderDtoBuilderSite.php</code> , <code>CreateOrderDtoBuilderMobile.php</code>
Валидация	<code>src/Modules/Order/Validators/</code> <code>CreateOrderCartSiteValidator.php</code> , <code>CreateOrderCartMobileValidator.php</code>
Основной сервис	<code>src/Modules/Order/Services/</code> <code>OrderService.php</code>
Побочные эффекты	<code>src/Modules/Order/AfterSaveEvent/</code> (26 классов)
Web-контроллер	<code>modules/orders/controllers/</code> <code>OrdersController.php</code>
Mobile-контроллер	<code>src/Modules/Mobile/Controllers/</code> <code>OrdersController.php</code>
Синхронизация с RCRM	<code>src/Modules/Rcrm/Service/</code> <code>RcrmOrderCreateService.php</code> , <code>RcrmOrderUpdateService.php</code>
Статусы для админки	<code>modules/statuses/</code>

Внимание: таблицу заказов обслуживают два ActiveRecord-класса. Это источник расхождений, см. [09-pitfalls.md](#).

Перечисления

```
// src/Modules/Order/Enum/StatusEnum.php – жизненный цикл заказа
RESERVED = 1, WAITING_PROCESSING = 3, PREPARING_PACKAGE = 4, WAITING_PAYMENT = 7,
AGREED_CLIENT = 8, PACKED = 11, ITEM_SENT = 14, AWAITING_PICKUP_IN_STORE = 16,
ORDER_COMPLETED = 17, ORDER_CANCELLED = 23, RESERVE_TIMED_OUT = 24, PREORDER = 35,
PREORDER_NEW = 60, PREORDER_PAYED = 61, CC_WAITING_FOR_PAYMENT = 70
ORDER_WAIT_PAYMENT_STATUSES = [7, 60, 70]

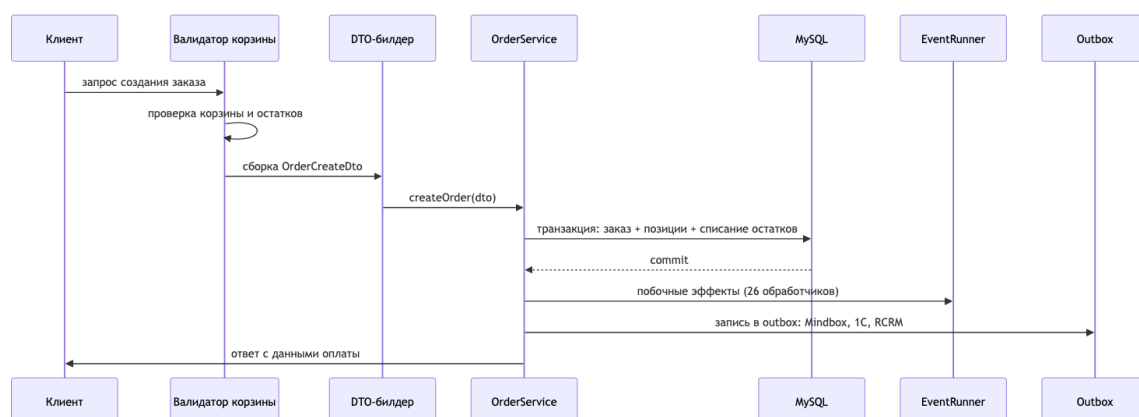
// PaymentStatusEnum – статус оплаты
SUCCESS = 1, NOT_PAID = 2, FAILURE = 3, CREDIT_APPROVED = 4

// OrderMethodEnum – канал создания
CART = 'shopping-cart', MOBILE = 'mobile', MOBILE_ANDROID = 'mobile-android',
PREORDER = 'preorder', MERCAUX = 'mercaux', LAMODA = 'lamoda',
BLOGGER = 'blogger', RETAIL = 'retailcrm'

// LastSourceEnum – кто последним менял заказ
CART, MOBILE, CMS, PAYMENT, YANDEX_SPLIT, YANDEX_PAY, YANDEX_SBP,
TBANK_DOLYAMI, ALPHA_PODELI, RETAIL, ONEC, AUTOCANCEL

// PaymentCategoryEnum – для правил BNPL
PRIMARY = 1, MODIFIED = 2, SURCHARGE = 3, TERMINAL = -1
```

Флоу создания заказа



Различия между каналами:

- **Сайт** — `modules/orders/controllers/OrdersController`, подготовка DTO через `CartOrderCreateDtoPrepareService`, при онлайн-оплате редирект на `/order/payment/{hash}`.
- **Мобильные приложения** — `CreateOrderServiceMobile`, ответ формирует `OrderCreateResponseTransformer` и включает `payment.send` (токен YooKassa) либо `payment_url`.

Обе ветки вызывают один и тот же `OrderService::createOrder()`.

Синхронизация с внешними системами

Исходящее направление — при создании заказа пишутся записи в outbox-таблицы, откуда воркеры отправляют их в RabbitMQ. Входящее — вебхуки и сообщения от RCRM и 1C обрабатывают `RcrmOrderCreateService` / `RcrmOrderUpdateService` и inbox-воркеры.

Платежи

Провайдеры

ID способов оплаты заданы в `src/Modules/Order/Enum/PaymentMethodEnum.php` (несмотря на название, класс лежит в модуле Order, а не Payment).

Провайдер	ID в <code>PaymentMethodEnum</code>	Где код
YooKassa (карта, сайт)	1, 6, 16	<code>modules/yandex/components/Yandex.php</code> , <code>controllers/PayController.php</code>
Tinkoff Dolyame	13	<code>modules/payment/components/Dolyame.php</code> , <code>controllers/TinkoffController.php</code>
Alpha Podeli	14	<code>src/Modules/Payment/Components/Podeli.php</code> (клиент), <code>modules/payment/controllers/AlphaPodeliController.php</code> (приём уведомлений — легаси-модуль)
Yandex Split	17	<code>src/Integration/Payment/YandexSplit/</code>
Yandex Pay	19	<code>src/Integration/Payment/YandexPay/</code>
Yandex SBP	20	<code>src/Integration/Payment/YandexSbp/</code>
Наличные, курьер, офлайн	2–5, 7, 8	без онлайн-провайдера
PayPal	9	легаси, в справочнике
Подарочный сертификат	15	<code>GiftcardsService</code>
Оплата при получении (резерв)	18	флоу резерва (<code>RESERVATION_POST_PAYMENT</code>)
Click & Collect Light	12	самовывоз из магазина

Групповые константы в `PaymentMethodEnum`: `BNPL_PAYMENT_METHODS`, `EXPRESS_PAYMENT_METHODS`, `DIRECT_ONLINE_PAYMENT_METHODS`, `ALL_ONLINE_PAYMENT_METHODS`, `PAYMENT_URL_AVAILABILITY_METHODS`.

ATOL — не платёжный провайдер, а фискализация чеков: `src/Modules/Receipt/Service/AtolService.php`, `Client/AtolClient.php`, легаси `modules/atol/`.

Паттерн outbox для платежей

Каждый провайдер имеет одинаковый набор компонентов:

1. таблица outbox-транзакций в MySQL;
2. таблица уведомлений;
3. обработчик транзакций (*TransactionHandler);
4. outbox-сервис (*OutboxService);
5. консольный контроллер уведомлений;
6. cron и consumer в Helm-values.

Центральный оркестратор — `src/Modules/Payment/Service/Outbox/OutboxTransactionService.php`, выбор провайдера — `src/Modules/Payment/Factory/PaymentProviderFactory.php`. Действия описаны `PaymentSystemOutboxActionEnum: commit, cancel, refund`.

При отмене заказа срабатывают события `SendCancelToDolyameEvent`, `SendCancelToPodeliEvent`, `SendCancelToSplitEvent`, `SendCancelToYandexPayEvent`, `SendCancelToYandexSbpEvent` — они лежат в модуле Order, а не Payment.

Приём уведомлений

Провайдер	Точка приёма
YooKassa	<code>modules/yandex/controllers/PayController.php</code>
Dolyame	<code>modules/payment/controllers/TinkoffController.php</code>
Alpha Podeli	<code>modules/payment/controllers/AlphaPodeliController.php</code>
Yandex (общий вебхук)	<code>src/Integration/Payment/Yandex/Controller/NotificationController.php</code>
Yandex Split / Pay / SBP	консольные <code>NotificationController</code> в соответствующих каталогах

Современные Yandex-провайдеры работают через клиентов биллингового микросервиса (`BillingMsClient`), а не напрямую.

Доставка

Область	Путь
Модели	<code>modules/delivery/models/</code> — <code>DeliveryType</code> , <code>DeliveryTk</code> , <code>DeliveryGroup</code> , <code>DeliveryCity</code> , <code>PickupPoint</code> , <code>BoxberryPoint</code>
Расчёт вариантов	<code>src/Modules/Delivery/Service/DeliveryVariantsService.php</code> , <code>DeliveryVariantBuilder.php</code>
Гео	<code>src/Modules/Delivery/Service/GeoService.php</code> , <code>LocationService.php</code>
Адреса	<code>modules/users/models/Address.php</code>
Самовывоз	<code>src/Modules/Delivery/Helper/PickupHelper.php</code>
Интервалы	через микросервис BFB (<code>MonolithBfbMsClient</code>)

```
// src/Modules/Delivery/Enum/OrderTypeEnum.php
ORDER = 0, CNC = 1, PREORDER = 2
```

Экспресс-доставка ограничена списком городов в `src/Modules/Cart/Enum/ExpressDeliveryEnum.php` (лежит в модуле Cart, а не Delivery) и лимитом позиций.

Дублирование: существуют два класса `PaymentTypeService` — в `modules/delivery/service/` и в `src/Modules/Delivery/Service/` — и используются они одновременно.

Пользователи и аутентификация

Область	Путь
Модель пользователя	<code>modules/users/models/User.php</code>
JWT	<code>src/Auth/Service/JwtService.php</code> , <code>AuthMethod/JwtHttpBearerAuthMethod.php</code>
Сервис авторизации	<code>src/Modules/Auth/Service/AuthService.php</code>
Мобильные сессии	<code>modules/mobile/models/UserMobileSession.php</code> , <code>src/Modules/Mobile/Services/MobileSessionService.php</code>
Легаси-токены API	<code>modules/api2/auth/HttpBearerAuth.php</code>
Бонусы	<code>modules/bonuses/models/Bonus.php</code> , <code>modules/mindbox/services/MindboxBonusService.php</code>
Подписки	<code>modules/subscription/</code> , <code>src/Modules/Subscription/</code>

Три независимых схемы аутентификации:

- **сайт** — сессия Yii + cookie;
- **API клиентов** — JWT на ES256, access + refresh, refresh хранится в БД;
- **мобильные приложения** — заголовки `x-access-token` (ключ приложения) и `x-session-token`.

Бонусы — источник истины Mindbox, локальные таблицы используются для отображения.

Подробнее об аутентификации API — [05-api.md](#).

Контент и CMS

Область	Путь
Ядро CMS	<code>modules/cms/</code> — параметры, i18n, админка, логи
Параметры-флаги	<code>modules/cms/models/Parameter.php</code> , обёртка <code>src/Modules/Cms/Model/Parameter.php</code>
Контентные страницы	<code>modules/content/</code> — <code>ContentController</code> , <code>AboutController</code> , <code>ShowroomsController</code>
Лендинги (легаси)	<code>modules/home/</code> — <code>LandingPageController</code> + PHP-шаблоны
Главная (современная)	<code>src/Modules/Mainpage/</code>
Баннеры и блоки	<code>modules/blocks/</code>
Лукбук	<code>modules/lookbook/</code> , <code>src/Modules/Lookbook/</code>
Блог, новости, вакансии, уход	соответствующие модули

Рекомендации и персонализация

Стратегия	Реализация
Оркестратор	<code>src/Modules/Product/Service/RecommendationsService.php</code>
По категории	<code>Recommendations/CategoryRecommendationsService.php</code>
Просмотренные	<code>Recommendations/LastViewRecommendationsService.php</code>
Mindbox	<code>Recommendations/MindboxRecommendationsService.php</code> , <code>MindboxRecommendationsForProductService.php</code>
Новинки	<code>Recommendations/NewProductRecommendationsService.php</code>
Страница «Спасибо»	<code>RecommendationsService::getThankYouPageRecommendations()</code>
Поиск и подсказки	<code>modules/diginetica/services/DigineticaService.php</code>
Подбор образов	<code>src/Modules/Feed/Service/GarderoboFeedService.php</code> + виджет на фронте

Выбор стратегии зависит от параметров-флагов (`recomendations_for_cart_category`, `IS_AVAILABLE_THANKYOU_RECOMMENDATIONS_*`).

Параметры и фича-флаги

Механизм включения функциональности без деплоя.

Компонент	Путь
Перечисление алиасов	<code>src/Modules/Common/Enum/Parameters.php</code> — 47 констант
Категории	<code>src/Modules/Common/Enum/ParameterCategoryEnum.php</code> — <code>cart</code> , <code>catalog</code> , <code>home</code> , <code>integrations</code> , <code>menu</code> , <code>orders</code> , <code>processing</code>
Хранение	таблица <code>parameters</code> , модель <code>modules/cms/models/Parameter.php</code>
Админка	<code>ParameterBackendController</code> с валидацией по JSON-схеме

Как работает:

1. флаг лежит в БД с полями `alias`, `value`, `value_type`, `category_id` и опциональной JSON-схемой;
2. код читает значение через `Parameter::valueOf($alias)`;
3. часто существуют платформенные варианты одного флага с суффиксами `_WEB`, `_IOS`, `_ANDROID`.

Новые флаги добавляются миграциями через трейт `ParameterManager` (методы вида `addBooleanParameter`).

Заметные флаги:

Флаг	Что включает
<code>express_delivery</code>	Экспресс-доставку
<code>early_access_enabled</code>	Ранний доступ к товарам
<code>sale_is_active</code> , <code>sale_timeframe</code>	Период распродажи
<code>promocode_apply_to_sale_products</code> , <code>promocode_max_percent</code>	Правила промокодов
<code>giftcards_allow_service*</code>	Работу с сертификатами
<code>new_checkout_client_payment_preselection</code>	Предвыбор способа оплаты
<code>min_order_sum_to_limit_online_payment</code>	Ограничение онлайн-оплаты по сумме
<code>max_minutes_wait_payment</code>	Таймаут ожидания оплаты
<code>is_dresscode_enabled</code> (константа <code>DRESSCODE_ENABLED</code>)	Интеграцию Dresscode
<code>web_notifications_enabled</code>	Веб-уведомления
<code>yandex_pay_sbp_surcharge_enabled</code>	Надбавку Yandex Pay SBP

Мобильные параметры дополнительно описаны в `src/Modules/Mobile/Enum/AndroidParametersEnum.php` и перечислениях `MobileV2`.

Важно: количество чтений (`valueOf` вызывается 233 раза в 111 файлах) заметно больше числа объявленных констант — часть флагов читается строковым алиасом мимо `enum`. См. [09-pitfalls.md](#).

Другие домены (кратко)

Список выше не исчерпывает бизнес-логику проекта. Ещё несколько доменов со своим объёмом кода, не разобранных подробно в этом документе:

Домен	Где код	Объём	Что делает
Отзывы (Feedback)	<code>modules/feedback/</code> (легаси, включает <code>console/controllers/dto/forms/repository</code>), <code>src/Modules/Feedback/</code>	70 + 13 файлов	Сбор и модерация отзывов о заказе/товаре/магазине; синхронизация через <code>Feedbacks\MobileClient</code> (gRPC-стаб, см. 05-api.md)
Синхронизация с 1С (OneC)	<code>src/Modules/OneC/</code> (Dto, Transformer, Enum, Controller, Configurator, Service, Client)	35 файлов	Обмен номенклатурой, остатками, заказами и сертификатами по SOAP; см. 08-integrations.md
A/B-тестирование	<code>modules/abTest/</code> (models, controllers, services, repositories)	16 файлов	Управление вариантами экспериментов через админку, таблицы <code>ab_test / ab_test_variant</code>
Резерв в магазине (Reservation)	<code>src/Modules/Reservation/</code> (Dto, Model, Service, Validator, Repository)	14 файлов	Отдельный домен для режима <code>MODE_RESERVATION</code> : <code>ReservationCartService</code> , <code>OrderReservationExpiration</code> , флоу оплаты при получении
Маркетинг	<code>modules/marketing/</code> (легаси), <code>src/Modules/Marketing/</code>	10 + 4 файла	Промо-механики, вспомогательные экстракторы
Wishlist (современный)	<code>src/Modules/Wishlist/</code>	3 файла	<code>WishlistService</code> , <code>WishlistController</code> — используется наравне с легаси <code>modules/wishlist/</code>

База данных

Общие сведения

MariaDB/MySQL, около 240 ActiveRecord-моделей с объявленным `tableName()`.

Префикс таблиц задаётся переменной `TABLE_PREFIX`, по умолчанию `cms_`. В моделях используется запись `{{%orders}}`, что в рантайме превращается в `cms_orders`.

Логи пишутся в отдельное подключение `log_db` (переменные `LOG_DSN`, `LOG_USERNAME`, `LOG_PASSWORD`), см. `config/common.php`.

Кэш схемы БД включён (`enableSchemaCache`, TTL 1 час — задаются в `config/common.php` и действуют для web и консоли), но хранится не в Redis, а в файлах: компонент `cacheSchema` типа `FileCache` в `@runtime/cache/dbSchema`. Сам компонент `cacheSchema` зарегистрирован только в `config/web.php` — в чистом `config/console.php` (без web-расширения) его нет.

Соглашения об именовании

- домены в именах таблиц идут после префикса: `catalog_*`, `orders_*`, `order_*`, `users_*`, `delivery_*`, `product_*`, `blog_*`;
- в моделях используется `{{%имя}}` для подстановки префикса;
- **исключения:** модели сертификатов задают имена жёстко — `cms_giftcard_sell`, `cms_giftcard_use`, `cms_giftcard2user` без `{{%}}`;
- таблица MySQL-очереди `default_queue` префикса не имеет в коде компонента — он подставляется из конфигурации.

Имена, которые отличаются от ожидаемых

Частый источник ошибок при написании запросов:

Ожидаемое имя	Фактическое
<code>cart</code>	<code>shopping_carts</code>
<code>cart_items</code>	<code>shopping_cart_positions</code>
<code>order_items</code>	<code>order_positions</code>
<code>product_colors</code>	нет такой таблицы; <code>products.color_id</code> → <code>dictionary_colors</code>
<code>order_positions.product_size_id</code>	<code>order_positions.item_id</code>

Ключевые таблицы по доменам

Заказы и чекаут

Таблица	Модель
orders	modules/orders/models/Order.php и src/Modules/Order/Model/Order.php
order_positions	modules/orders/models/OrderPosition.php
orders_history	modules/orders/models/OrderHistory.php
order_positions_history	modules/orders/models/OrderPositionHistory.php
orders_payments	modules/payment/models/Payment.php
order_payment_links	src/Modules/Payment/Models/OrderPaymentLink.php
order_errors	modules/orders/models/OrderError.php
orders_exchange_log	modules/orders/models/OrdersExchangeLog.php
order_reservation_expiration	src/Modules/Reservation/Model/OrderReservationExpiration.php
offline_orders , offline_order_positions	src/Modules/Order/Model/OfflineOrder*.php
offline_order_retry_queue	src/Modules/Order/Model/OfflineOrderRetryQueue.php
shopping_carts	modules/cart/models/Cart.php
shopping_cart_positions	modules/shopping/models/ShoppingCartPosition.php
checkout_user_settings	src/Modules/Cart/Model/CheckoutUserSettings.php
wishlist_items	modules/wishlist/models/WishlistItem.php

Товары и каталог

Таблица	Модель
products	modules/product/models/Product/Product.php
product_sizes	modules/product/models/ProductSize.php
product_category	modules/product/models/ProductCategory.php (составной ключ)
dictionary_colors	modules/product/models/Colors/DictionaryColor.php
dictionary_sizes	modules/product/models/DictionarySize.php
catalog_category	modules/catalog/models/Category.php
catalog_category_tree_items	src/Modules/Product/Models/CategoryTreeItem.php (Nested Sets)
catalog_stock	modules/catalog/models/Stock.php — склады и магазины
catalog_item_stock	modules/catalog/models/ItemStock.php — остаток по штрихкоду и магазину
product_stocks	src/Modules/Product/Models/ProductStock.php
product_attributes	src/Modules/Product/Models/ProductAttribute.php
product_order_kate , product_order_sonya	веса мерчандайзинговой сортировки

Пользователи

Таблица	Модель
users_user	modules/users/models/User.php
users_address	modules/users/models/Address.php
users_auth	modules/users/models/Auth.php
users_geo	src/Modules/Delivery/Model/UserGeo.php
users_mobile_sessions	modules/mobile/models/UserMobileSession.php
jwt_refresh_token	src/Auth/Model/JwtRefreshToken.php
user_import_files	src/Modules/User/Models/UserImportFile.php

Платежи и чеки

Таблица	Назначение
orders_payments	Платежи по заказу
payment_types , payment_statuses	Справочники
receipts , order_receipts	Фискальные чеки
order_receipts_to_send , *_outbox	Асинхронная отправка чеков
dolyame_payment_transaction_outbox	Outbox Dolyame
podeli_payment_transaction_outbox	Outbox Podeli
split_payment_transaction_outbox	Outbox Yandex Split
yandex_pay_payment_transaction_outbox	Outbox Yandex Pay
yandex_sbp_payment_transaction_outbox	Outbox Yandex SBP
notification_podeli , notification_yandex_split , notification_yandex_pay , notification_yandex_sbp	Входящие уведомления провайдеров

Доставка и гео

Таблица	Назначение
delivery_tk	Транспортные компании
delivery_types , delivery_city , delivery_ipgeo	Справочники доставки
geo_city	Города (src/Modules/Delivery/Model/ GeoCity.php)
pickup_points , boxberry_point , accord_point	Пункты выдачи
return_method , return_details_config	Конфигурация возвратов

CMS и флаги

Таблица	Назначение
parameters , parameters_content , parameter_categories	Фича-флаги и настройки
mainpages , mainpage_blocks , mainpage_block_elements	Главная страница
ab_test , ab_test_variant	A/B-тесты
events_toggle	Переключатели аналитических событий
notifications	In-app уведомления пользователей

Интеграционные outbox и inbox

Направление	Таблицы
RetailCRM	<code>rcrm_create_transaction_outbox/inbox</code> , <code>rcrm_update_transaction_outbox/inbox</code>
1C	<code>onec_create_transaction_outbox</code> , <code>onec_update_transaction_outbox/inbox</code>
Mindbox	<code>mindbox_create_transaction_outbox</code> , <code>mindbox_update_transaction_outbox</code>
Lamoda	<code>lamoda_orders_outbox</code>
Фиды	<code>feed_counters</code>

Связи ключевых сущностей

Корзина

```
shopping_carts
├─ user_id → users_user.id
├─ hasMany shopping_cart_positions (cart_id)
│   └─ product_id → products.id
│   └─ product_size_id → product_sizes.id
```

Позиции корзины загружаются через `getPositionRelations()` с `joinWith('productRelation')` — без N+1.

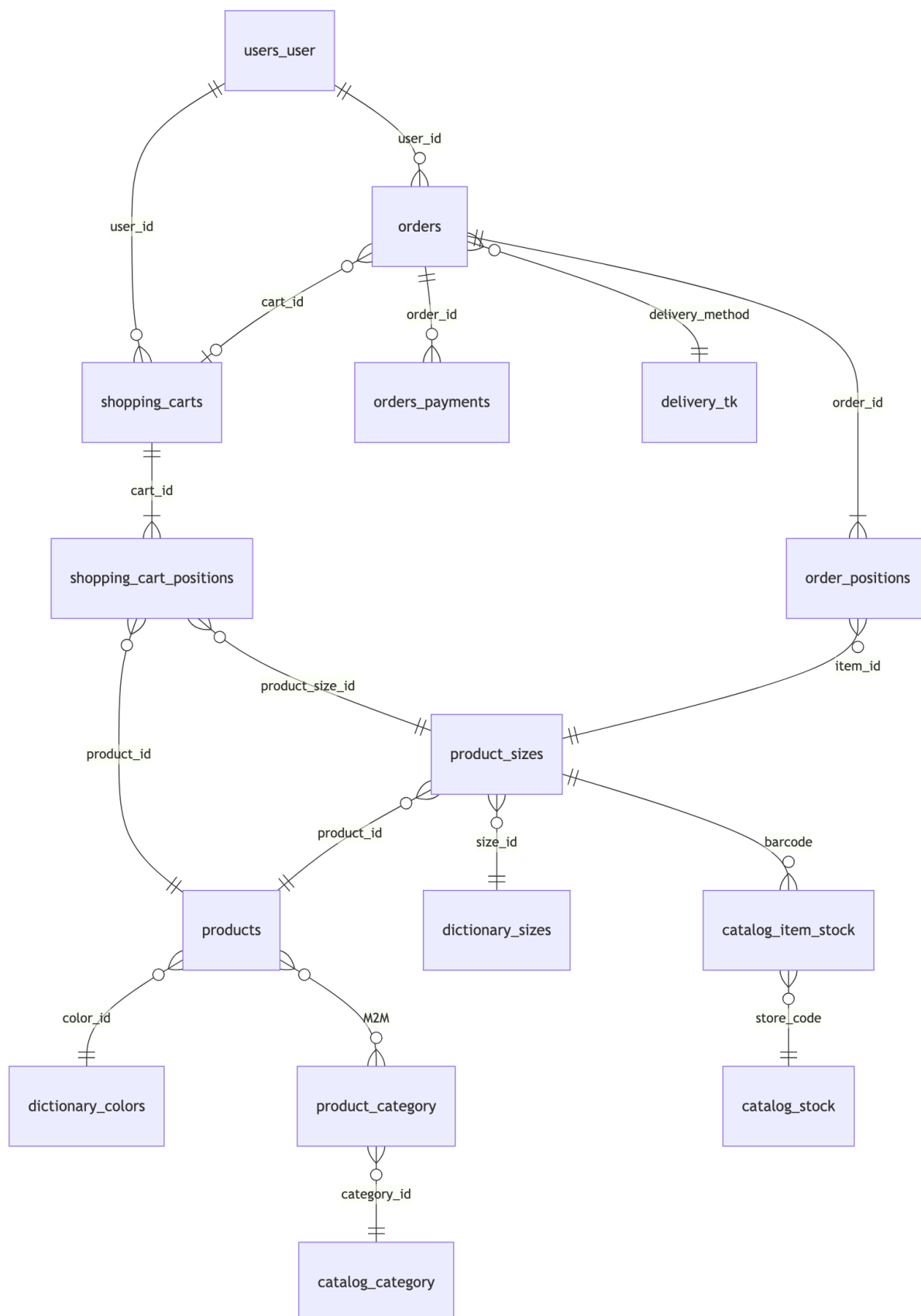
Заказ

```
orders
├─ user_id → users_user.id (relation getUser())
├─ cart_id → shopping_carts.id (поле есть, но relation getCart() в моделях Order нет — связь только через прямой запрос по cart_id)
├─ address_id → users_address.id (relation getAddress())
├─ delivery_method → delivery_tk.id (relation getDeliveryMethod())
├─ hasMany order_positions (order_id)
│   └─ item_id → product_sizes.id ← не product_size_id!
│   └─ товар получается через размер
│   └─ stock_id → catalog_stock.id
│   └─ status → order_position_statuses
├─ hasMany orders_payments (order_id)
└─ hasOne order_statuses (no internal_id)
```

Товар

```
products
├── color_id      → dictionary_colors.id
├── main_category_id → catalog_category.id
├── категории     → JSON-поле + связь M2M через product_category
├── hasMany product_sizes (product_id)
│   ├── size_id → dictionary_sizes.id
│   ├── hasMany catalog_item_stock (no barcode)
│   └── hasOne  product_stocks (no barcode)
└── цветковые варианты – отдельные строки products с общим group_guid / super_model_guid
```


ER-диаграмма коммерческого ядра



Служебные и легаси-таблицы

Таблица	Назначение	Примечание
<code>default_queue</code>	MySQL-очередь заданий	Колонки <code>queue</code> , <code>run_at</code> , <code>payload</code> , <code>ukey</code> . Имя жестко захардкожено в <code>components/SqlQueue.php</code> (<code>SqlQueueName::DEFAULT</code> . <code>'_queue'</code>) — <code>TABLE_PREFIX</code> к нему не применяется, в отличие от моделей ActiveRecord
<code>slave_queue</code>	Стейджинг синхронизации заказов	<code>modules/cms/models/SlaveQueue.php</code>
<code>log</code> , <code>frontend_logs</code>	Логи приложения и фронтенда	Отдельное подключение <code>log_db</code>
<code>product_order_kate</code> , <code>product_order_sonya</code>	Веса сортировки каталога	Пересобираются через временные таблицы
<code>product_size_stocks_temp</code>	Стейджинг импорта остатков	TRUNCATE + массовая загрузка
<code>catalog_item_quantity_buffer</code>	Буфер остатков под заказы	
<code>catalog_item_stocks_quantity_buffer</code>	Буфер синхронизации остатков	<code>src/Modules/Stock/Model/ItemStockQuantityBuffer.php</code>
<code>low_stock_alert_state</code>	Однократные алерты о низком остатке	
<code>complete_messages</code> , <code>porpmechanic_contacts</code>	Маркетинговые артефакты	

Остаток товара получается из трёх источников (`catalog_item_stock`, `product_stocks`, буферы) и согласован не мгновенно, а по мере обработки очередей.

Миграции

196 файлов, диапазон: `m250403_150841_5479_add_is_express_to_cart.php` (апрель 2025) — `m260817_093503_add_index_item_stock_store_quantity_catalog_item_stock_table.php` (август 2026). **92 миграции (47%) написаны в 2026 году.**

Соглашение об именовании

```
m{ГГММДД}_{ЧЧММСС}_{номер задачи}_{описание_через_подчёркивания}.php
```

Примеры:

- `m260602_100002_add_idx_orders_cart_id.php` — суффикс времени используется для упорядочивания связанных миграций одного дня;
- `m250403_150841_5479_add_is_express_to_cart.php` — с номером задачи;

- `m260210_091507_ab_test_ecomdev_887.php` — со ссылкой на задачу в описании.

Флаги-параметры добавляются через трейт `ParameterManager` (`addBooleanParameter` и аналоги).

Распределение по доменам

Классификация приблизительная — часть миграций затрагивает несколько доменов сразу (например, добавление параметра, влияющего и на корзину, и на доставку) и отнесена к основному контексту по смыслу имени файла. Строка «Прочее» — разница между суммой явно классифицированных строк и общим числом миграций (196 / 92), а не отдельная категория.

Домен	Всего	Из них 2026
Заказы, корзина, чекаут	47	27
Доставка, гео, ФИАС	34	8
Параметры CMS, мобильные, A/B	33	19
Товары, каталог, остатки	30	18
Платежи, сертификаты, BNPL	29	15
Пользователи, адреса	8	3
Прочее / на стыке доменов	15	2
Итого	196	92

Что разрабатывалось в 2026 году

По кластерам миграций видно направление работ:

- Резерв в магазине** (март–апрель) —
`m260323_181900_1106_add_reservation_delivery_type.php` ,
`m260407_090042_create_reservation_parameters.php` ,
`m260417_150054_1339_add_order_reservation_expiration_table.php` .
- Новый чекаут и предвыбор оплаты** (март–июнь) —
`m260330_143915_add_payment_method_preselection_parameter.php` ,
`m260331_083825_add_new_checkout_abc_enum_parameter.php` ,
`m260617_113742_add_preselected_payment_feature_parameter.php` .
- Yandex Pay и Yandex SBP** (май–июль) —
`m260512_121100_add_new_yandex_pay_payment_type.php` ,
`m260520_121200_add_yandex_pay_fields_to_orders.php` ,
`m260625_170000_add_yandex_sbp_payment_method.php` ,
`m260701_130000_add_yandex_sbp_outbox_notification_and_parameters.php` .
- Ранний доступ** (июль) — `m260728_120000_add_early_access_parameter.php` ,
`m260728_130000_add_early_access_dates_to_products_table.php` ,
`m260729_140000_add_qty_site_early_access_to_product_stocks_table.php` .
- In-app уведомления** (июль) — `m260713_111620_create_notifications_table.php` ,
`m260804_140914_add_parameter_web_notifications_enabled.php` .

6. **Индексы производительности** (июнь) — серия `m260602_100001 ... m260602_100004` на `orders.order_num`, `orders.cart_id`, `wishlist_items._created` (поле называется с ведущим подчёркиванием), `shopping_carts.updated_at`.
7. **Повторные попытки офлайн-заказов и флаги RCRM** (апрель–июль) — `m260420_122824_add_offline_order_retry_queue_table.php`, `m260730_065013_add_is_rcrm_modified_column_order.php`.

Проверка парных миграций истории в CI

Скрипт `check-migrate-trigger.sh` запускается в CI и требует, чтобы миграция, меняющая `{%orders}`, `{%order_positions}` или `{%products}`, шла в паре с миграцией соответствующей таблицы истории (`{%orders_history}`, `{%order_positions_history}`, `{%product_history}`).

У скрипта есть дефект — он корректно работает только при одной изменённой миграции. Подробности в [09-pitfalls.md](#). Правило при этом реальное: **изменяя основные таблицы, добавляйте миграцию для таблицы истории.**

Способы доступа к данным

Слой	Где	Когда используется
ActiveRecord (легаси)	<code>modules/*/models/</code>	Корзина, заказ, товар — с бизнес-логикой в модели
ActiveRecord (современный)	<code>src/Modules/*/Model/</code>	Outbox, уведомления, резерв
Репозитории (~190)	<code>src/Modules/*/Repository/</code> , <code>modules/*/repository/</code>	Предпочтительно для нового кода
Сырой SQL	Репозитории, миграции, консольные команды	Массовый обмен остатками, пересборка сортировок, очистка outbox

Примеры тяжёлых запросов:

Файл	Что делает
<code>modules/api2/repository/ExchangeStockRepository.php</code>	Многотабличные JOIN, TRUNCATE временной таблицы, массовый INSERT
<code>modules/product/repository/ProductOrderSonyaRepository.php</code>	CREATE TEMPORARY TABLE и UPSERT в таблицу сортировки
<code>src/Modules/Order/Repository/MindboxOrderUpdateOutboxRepository.php</code>	Дедупликация через DELETE с self-JOIN
<code>modules/payment/repository/ReceiptRepository.php</code>	INSERT ... ON DUPLICATE KEY UPDATE
<code>modules/catalog/repository/CatalogDataSource.php</code>	Загрузка дерева категорий с последующим кэшированием в Redis

Известные места с риском N+1

- `ProductSize::getSize()` вызывает `hasOne(...)->one()` внутри метода, а не через СВОЙСТВО-СВЯЗЬ.
- `OrderPosition::beforeSave()` подгружает товар и размер на каждое сохранение.
- Фиды: в `YandexFeedService` оставлен комментарий о необходимости заранее загружать товары с `with('sizes')`.

Поиск

Поиск работает на **Elasticsearch**: `src/Search/Elastic/`, индексатор `ElasticCatalogIndexerService`, алиас индекса задаётся параметром `elastic_catalog_index`.

Переиндексация идёт через MySQL-очереди (`reindexproduct`, `reindexproductstocks`, `reindexproductsizes`) и RabbitMQ.

Пакет `yiisoft/yii2-sphinx` и файл `config/sphinx.php` присутствуют, но в **рантайме не используются** — компонент не зарегистрирован, импортов в коде нет.

Redis

Доступ через `Application\Cache\Redis\Contract\RedisInterface` с обязательным указанием группы: `$redis->byGroup(RedisGroupDictionary::GROUP_*)`. Групп 40.

Основные группы:

Область	Группы
Каталог	<code>CatalogCategories</code> , <code>BlockCategories</code> , <code>FastProduct</code> , <code>ProductsMobileListv2</code> , <code>CategoryDefaultProductsList</code>
Остатки	<code>stocks</code> , <code>ProductStocksMobile</code>
Корзина и чекаут	<code>NewCheckout</code> , <code>OrderCreateAction</code> , <code>PaymentType</code>
Мобильные	<code>UserMobileSession</code> , <code>MobileCategories</code> , <code>MobileInfo</code> , <code>AppVersionsConfig</code>
Интеграции	<code>Integration</code> , <code>Mindbox</code> , <code>Webhook0nec</code> , <code>Elastic</code>
Прочее	<code>AbTests</code> , <code>Diginetica</code> , <code>SessionLocation</code> , <code>Auth</code> , <code>ActionPresent</code>

Группы, исключённые из массового сброса (`getFlushExcludedGroups()`): `Logs`, `ExchangeErrors`, `Auth`, `Integration`, `OrderCreateAction`, `Webhook0nec`, а также `RateLimiter` (`RateLimiterConfigurator::REDIS_GROUP`).

Помимо этого Redis используется как бэкенд Yii-компонентов кэша (`cache`, `cacheMain`, `cacheYiiT`). Формирование ключа имеет неочевидную особенность — см. [07-infrastructure.md](#) и [09-pitfalls.md](#).

Хранение сессий:

- **корзина на сайте** — `shopping_carts.session_id` (не Redis);
- **мобильная авторизация** — таблица `users_mobile_sessions` с кэширующей обёрткой;
- **JWT** — таблица `jwt_refresh_token` плюс denylist в Redis с префиксом `jwt:deny:` .

API

Три генерации API

В системе одновременно работают три поколения API с разными схемами аутентификации и разными принципами построения ответа.

Генерация	Префикс URL	Аутентификация	Кто использует
Партнёрские (легаси)	/api/*, /api2/*	Токен в query или Bearer	1C, RetailCRM, Lamoda, Mercaux, WMS
Web (современная)	/api/auth/*, /api/cart/*, /api/v1/*	JWT ES256	Сайт, новый чекаут
Mobile	/mobile/*, /mobile/v2/*	Ключ приложения + токен сессии	iOS и Android

Отдельно стоят серверные страницы каталога, отдающие как HTML, так и JSON (/product/product/*-json), и админка (/cms/*, 127 контроллеров *BackendController).

Маршрутизация

Правила описаны в config/url.php (336 строк). Порядок правил важен — срабатывает первое совпадение. Помимо обычных правил используются кастомные классы UrlRule :

Класс	Файл	Что обслуживает
Application\Auth\Rule\UrlRule	src/Auth/Rule/UrlRule.php	/api/auth/*
Application\User\Rule\UrlRule	src/User/Rule/UrlRule.php	/api/user/me
Application\User\Notifications\Rule\UrlRule	src/User/Notifications/Rule/UrlRule.php	/api/v1/notifications/*
Application\Modules\Mobile\Rules\UrlRule	src/Modules/Mobile/Rules/UrlRule.php	REST-пути mobile v1 с ID в URL
modules\mobilev2\rules\UrlRule	modules/mobilev2/rules/UrlRule.php	REST-пути mobile v2
modules\product\rules\UrlRule	modules/product/rules/UrlRule.php	SEO-адреса каталога
modules\content\rules\UrlRule	modules/content/rules/UrlRule.php	Статические страницы

В конце файла стоят общие правила <module>/<controller>/<action> , которые перехватывают всё несовпавшее.

Платёжный вебхук Yandex зарегистрирован отдельно через `controllerMap` в `config/web.php` :
`POST /yandex/notification/v1/webhook` .

Особенность mobile v2

Префикс `/mobile/v2/` обслуживают **два модуля одновременно**:

- `modules/mobilev2/` — основной трафик, работает через общее правило;
- `src/Modules/MobileV2/` — новые эндпоинты по явным правилам (лукбуки, настройки, YooKassa, смена способа оплаты, лендинги, гео пользователя).

Куда попадёт запрос, определяет порядок правил. Подробнее — [09-pitfalls.md](#).

Аутентификация

Web: JWT

Реализация на `lcobucci/jwt` , алгоритм ES256.

- Ключи читаются из переменных окружения `JWT_PRIVATE_KEY` и `JWT_PUBLIC_KEY` (в base64). Каталог `jwt_keys/` в репозитории **не используется кодом**.
- `src/Auth/Service/JwtService.php` выдаёт access- и refresh-токены.
- Refresh-токены хранятся в таблице `jwt_refresh_token` , отозванные access-токены — в Redis-denylist с префиксом `jwt:deny:` .
- Фильтр `JwtHttpBearerAuthMethod` подключается к контроллерам; для эндпоинтов входа он необязателен.

Публичные эндпоинты: `send-code` , `login` , `login-by-email` , `refresh` . Защищённые: `logout` , `/api/user/me` , `/api/cart/*` , уведомления, чекаут.

Mobile: ключ приложения плюс сессия

Все мобильные контроллеры наследуют `ApiBaseController` / `BaseController` и требуют заголовки:

Заголовок	Назначение
<code>x-access-token</code>	Ключ приложения, сверяется с <code>MOBILE_API_KEY</code>
<code>x-session-token</code>	Сессия (гостевая или пользовательская), таблица <code>users_mobile_sessions</code>
<code>x-platform</code>	Платформа
<code>x-version</code>	Версия приложения

Отдельное свойство `isAuthRequired` в контроллере определяет, нужен ли действию привязанный к сессии пользователь.

Создание сессии: `PUT /mobile/user/session` (гостевая), `PUT /mobile/user/token` (обмен учётных данных). Вход: `POST /mobile/user/auth` , `/mobile/user/auth-phone` .

Есть список заблокированных устройств — переменная `MOBILE_DEVICE_ID_BAN_LIST`.

Партнёрские интеграции: токены

Токены хранятся в модели `modules/api/models/Token`. В `api v1` передаются query-параметром `?token=`, в `api2` — либо query-параметром, либо Bearer-заголовком (`modules/api2/auth/HttpBearerAuth.php`).

Сводка по гостевому доступу

Поверхность	Гость	Авторизованный
Каталог сайта (JSON)	доступен	добавляется персонализация
<code>/api/cart/*</code>	корзина по сессии	JWT добавляет данные пользователя
<code>/api/auth/*</code>	вход, отправка кода, refresh	logout и защищённые ресурсы
Mobile	гостевая сессия	сессия с привязанным пользователем
<code>/api/*</code> , <code>/api2/*</code>	только по валидному токenu интеграции	—

Эндпоинты

Аутентификация и пользователь

Метод и путь	Обработчик	Назначение
POST <code>/api/auth/send-code</code>	<code>AuthController::actionSendCode</code>	Отправка SMS-кода (капча + rate limit)
POST <code>/api/auth/login</code>	<code>AuthController::actionLogin</code>	Вход по телефону и коду
POST <code>/api/auth/login-by-email</code>	<code>AuthController::actionLoginByEmail</code>	Вход по email и паролю
POST <code>/api/auth/refresh</code>	<code>AuthController::actionRefresh</code>	Обновление токенов
POST <code>/api/auth/logout</code>	<code>AuthController::actionLogout</code>	Отзыв access-токена
GET <code>/api/user/me</code>	<code>UserController::actionMe</code>	Профиль текущего пользователя

Корзина — сайт

Контроллеры в `src/Modules/Cart/Controllers/`.

Метод и путь	Обработчик
GET /api/cart/info	CartApiController::actionGetInfo
GET /api/cart/items	CartApiController::actionGetItems
PATCH /api/cart/items	CartApiController::actionUpdateItem
DELETE /api/cart/items	CartApiController::actionDeleteItem
GET / POST /api/cart/user-info	CartApiController::actionGetUserInfo / actionSaveUserInfo
POST /api/cart/move-unavailable-to-favourites	CartApiController::actionMoveUnavailableToFavourites
GET / POST /api/cart/delivery	CartDeliveryController::actionDelivery / actionSaveDelivery
GET /api/cart/delivery-list	CartDeliveryController::actionDeliveryList
GET / POST /api/cart/delivery-city	CartDeliveryController::actionCity / actionSaveCity
GET / POST /api/cart/delivery/pvz	CartDeliveryController::actionGetDeliveryPvz / actionSavePvz
GET / POST /api/cart/delivery/pickup-points	CartDeliveryController::actionGetList / actionSavePickupPoint
GET /api/cart/payment	CartPaymentController::actionIndex
GET /carts/main/count	CartController::actionCount

Корзина — mobile v1

Контроллеры в `src/Modules/Mobile/Controllers/`.

Метод и путь	Обработчик
GET / PUT /mobile/cart	CartController::actionIndex
GET /mobile/cart/info	CartDetailsController::actionInfo
GET / PATCH / DELETE /mobile/cart/items	CartDetailsController::actionItems
PATCH / DELETE /mobile/cart/<barcode>	CartController::actionUpdate (не /mobile/cart — там только actionIndex на GET/PUT)
POST /mobile/cart/apply-order-settings	CartDetailsController::actionApplyOrderSettings
GET / POST /mobile/cart-delivery/*	CartDeliveryController
GET / POST /mobile/cart-payment/*	CartPaymentController
GET / POST / DELETE /mobile/cart-giftcard/*	CartGiftcardController
POST / DELETE /mobile/cart-discount/promo-code	CartDiscountController::actionPromoCode (не PUT)
GET / POST /mobile/cart-discount/bonuses	CartDiscountController::actionBonuses (не PUT / DELETE — DELETE не реализован)
POST /mobile/cart/payment	CartController::actionPayment

Заказы

Метод и путь	Обработчик
POST / order/ proceed- new	modules/orders/.../OrdersController::actionProceedNew
GET / order/ payment/ <hash>	OrdersController::actionPayment
GET / POST / mobile/ orders	Mobile\OrdersController::actionIndex
GET / mobile/ orders/ {id}	Mobile\OrdersController (через UrlRule)
POST / mobile/ orders/ cancel	Mobile\OrdersController::actionCancel
POST / mobile/ orders/ return- positions	Mobile\OrdersController::actionReturnPositions (id — query-параметром, метод POST, не GET)
POST / mobile/ orders/ {id}/ return- details	Mobile\OrdersController::actionReturnDetails
GET /api/ orders/ {id}/ return- method- list	Application\Modules\Api2\Controller\OrdersController::actionReturnMethodList (модуль api2_new, не легаси api2)
PATCH / api/ orders/ {id}/ change- payment- method	Payment\Controller\ChangePaymentController (только PATCH ; GET списка типов оплаты — отдельный эндпоинт GET /api/payment-types)
PATCH / mobile/ v2/	MobileV2\Controllers\ChangePaymentController (только PATCH ; GET — GET /mobile/v2/payment-types)

Метод и путь	Обработчик
orders/{id}/change-payment-method	

Каталог и товары

Метод и путь	Обработчик
GET /catalog/{slug}	ProductController::actionIndex (через UrlRule)
GET /product/product/index-json	ProductController::actionIndexJson
GET /product/product/count	ProductController::actionCountJson (URL без -json, метод — c)
GET /catalog/{slug1}/{slug2}/{slug3}	ProductController::actionView (карточка товара по SEO-адресу, не /product/product/view)
GET /catalog/search	ProductController::actionSearchProduct
GET /catalog/suggest	ProductController::actionSuggestJson
GET /catalog/all-fashion-json	ProductController::actionAllFashionJson (URL резолвится в action-id all-fashion, который формально не совпадает с именем метода — см. 09-pitfalls.md)
GET /product/product/recommendations	ProductV2\ProductController::actionRecommendations
GET /api/v1/get-url-by-article/{article}	ProductV2\ProductController::actionGetUrlByArticle
GET /mobile/products/{id}/last-search, /fashion/{id}, /stocks/{id}, /accompaniments/{id}, /last-view/{id}	Mobile\ProductsController — методы actionLastSearch, actionRecommendations, actionStoreAvailability, actionProductRecommendations, actionCompleteSet (у контроллера нет actionIndex, поэтому голого GET /mobile/products/{id} не существует)
GET /mobile/v1/products/{id}/recommendations	Mobile\ProductsController::actionProductRecommendations
GET /mobile/v2/products/{id}	mobilev2\ProductsController::actionIndex
GET /mobile/category/list	Mobile\CategoryController::actionList
GET /mobile/catalog/filter	Mobile\CatalogController::actionFilter
GET /mobile/v2/lookbooks	MobileV2\LookbookController::actionList

Доставка и гео

Метод и путь	Обработчик
GET /api/v1/delivery/geo/cities	DeliveryV2\GeoController::actionCities
GET /api/v1/delivery/geo/countries/{iso}/addresses	GeoController::actionSuggestAddress
GET /api/v1/delivery/pickup/provider/{id}/points	PickupController::actionPointListJson
GET /mobile/delivery	Mobile\DeliveryController::actionIndex
GET /mobile/delivery/intervals	Mobile\DeliveryController::actionIntervals
POST /mobile/v2/geo/confirmation	MobileV2\UserGeoController::actionConfirmation

Партнёрские API

Метод и путь	Обработчик	Назначение
GET / POST / api/clients	api\ClientsController	Пользователи для RCRM
GET / POST / PATCH / DELETE / api/ products/ {id}	api\ProductsController	Синхронизация товаров
GET / POST / api/orders	api\OrdersController	Синхронизация заказов
GET /api/ orders/a- payments/ {begin}/ {end}	api\OrdersController::actionPayments	Выгрузка платежей
GET /api2/ healthcheck/ ping	HealthcheckController::actionPing	Проверка живости
GET /api2/ products/ items	api2\ProductsController::actionItems	Товары для партнёров
POST /api2/ orders/ create	api2\OrdersController::actionCreate	Создание заказа (Lamoda)
POST /api2/ orders/ order- update	api2\OrdersController::actionOrderUpdate	Обновление от WMS
POST /api2/ orders/ status	api2\OrdersController::actionStatus	Смена статуса
GET /api2/ stocks	api2\StocksController::actionIndex	Снимок остатков
POST /api2/ stocks/esb	Application\Modules\Api2\Controller\StocksController::actionEsb (роутится на api2_new , а не легаси modules/api2)	Приём остатков из ESB
POST /api2/ popmechanic/ contact	PopmechanicController::actionContact	Контакт в CRM

В api v1 действуют два стиля маршрутов: api/<controller>/a-<action> и REST-варианты.

Формирование ответа

Единого сериализатора для публичного API нет. Ответ собирают **экстракторы** и **трансформеры**.

Экстракторы реализуют `ExtractorInterface` и превращают модели в массивы. Под каждый канал — свой экстрактор:

Экстрактор	Назначение
<code>ProductItemFullDtoExtractor</code>	Полная карточка товара
<code>ProductFullMobileV2Extractor</code>	Карточка для mobile v2
<code>ProductListMobileV2Extractor</code>	Список товаров для mobile v2
<code>ProductListMobileV1Extractor</code>	Список для mobile v1
<code>ProductListApi2Extractor</code>	Список для партнёров
<code>AddressMobileV2Extractor</code>	Адреса пользователя

Трансформеры используются в новых модулях: `src/Modules/Cart/Transformer/`, `src/Auth/Transformer/`, `src/Modules/MobileV2/Transformer/`.

Форматы обёртки

Mobile v1 — через `ApiBaseController::answer()`:

```
{ "success": true, "total": 10, "result": [] }
```

При ошибке:

```
{ "result": { "error_code": 1001, "error_text": "..." } }
```

Web JWT — через `FrontendApiResponseService` (успех и ошибка — разные наборы полей, ключ называется `status`, а не `success`):

```
{ "status": "success", "message": "", "data": {} }
```

При ошибке:

```
{ "status": "error", "message": "", "code": "SERVER_ERROR", "errors": [] }
```

Дополнительно сервис добавляет заголовки `X-Response-Id`, `X-Response-Version`, `X-Language` к каждому ответу.

Следствие: один и тот же товар отдаётся в разных формах для сайта, mobile v1, mobile v2 и партнёров. Добавляя поле, нужно решить, в каких экстракторах оно должно появиться.

Валидация

Инструмент	Где применяется
Symfony Validator	DTO в модулях Cart, Feedback, вебхуки 1С и доставки — через <code>DtoValidator</code>
Opis JsonSchema	Схемы параметров CMS (<code>ParameterSchemaValidator</code>), обмен товарами (<code>ExchangeSchemaValidator</code>)
Валидаторы Yii и кастомные	Авторизация (<code>RequestLoginValidator</code>), правила полей в <code>ApiBaseController</code>
MobileV2	<code>AppVersionValidator</code> и исключения корзины

Symfony Serializer используется ограниченно — для разбора входящих вебхуков и в консольных обработчиках платежей, но не для формирования публичных ответов.

Контракты и документация

Спецификации OpenAPI:

Файл	База	Путей	Операций	operationId	Область
docs/mobileApi/openapi.yml	https://12storeez.com/mobile	113	138	138	Mobile v1 и v2
docs/frontend/openapi.yml	https://12storeez.com/	144	152	114	Web: авторизация, корзина, каталог
docs/ssr/openapi.yml	https://12storeez.com/ssr/api/	21	21	5	Авторизация, вишлист, меню

У frontend и ssr спецификаций часть операций не имеет operationId — на подсчёт покрытия это не влияет, так как проверка ниже вообще не использует operationId.

Покрытие проверяется в CI скриптом `openapi-coverage.php` по более грубой логике: он суммирует количество методов `action*` (во всех файлах `*Controller.php` внутри `controllers/` -подобных каталогов `modules/**` и `src/**` (исключая только `*BackendController.php`, абстрактные классы не считаются отдельно) и делит на суммарное число HTTP-операций (`get / post / put / patch / delete`) во **всех** файлах `docs/**/openapi.yml` — сопоставления конкретного action с конкретной операцией нет, сравниваются только общие числа. **Минимальный порог — 50%** (`OPENAPI_COVERAGE_MIN_PERCENTAGE`).

Вебхуки и входящие интеграции

HTTP-вебхуки

Метод и путь	Обработчик	Источник
POST /yandex/ notification/ v1/webhook	Integration\Payment\Yandex\Controller\NotificationController	Yandex Pay / Split
POST / payment/ alpha-podeli/ notification	modules/payment/controllers/ AlphaPodeliController::actionNotification	Alpha Podeli
POST /api/ onec/ webhooks/ change- balance	OneC\WebhooksController::actionChangeBalance	1С, баланс сертификата
POST /api/ onec/ webhooks/ change- status	OneC\WebhooksController::actionChangeStatus	1С, статусы
POST /api/ onec/ webhooks/not- web-purchase	OneC\WebhooksController::actionNotWebPurchase	1С, офлайн-покупка
POST /api/ onec/ webhooks/ change-owner	OneC\WebhooksController::actionChangeOwner	1С, смена владельца сертификата
POST /api/v1/ delivery/ webhooks/ city- carriers- coverage	DeliveryV2\WebhooksController	Логистика
POST /api2/ orders/ status	api2\OrdersController::actionStatus	WMS / RCRM
POST /api2/ orders/ notification- from-lamoda	api2\OrdersController::actionNotificationFromLamoda	Lamoda

Консольные консьюмеры

Обработчик	Назначение
<code>src/Integration/Payment/Yandex*/Console/NotificationController.php</code>	Уведомления Yandex Pay / Split / SBP
<code>src/Modules/Payment/Console/NotificationController.php</code>	Обработка платёжного outbox
<code>src/Modules/Order/Controller/RabbitInboxTransactionController.php</code>	Входящие изменения заказов
<code>src/Modules/Notifications/Console/NotificationsController.php</code>	Отправка уведомлений

gRPC

`modules/grpc/` содержит сгенерированные PHP-клиенты (файлы `.proto` в репозитории не хранятся). Клиенты регистрируются в DI, адреса и ключи берутся из параметров.

Сервис	Клиент	Методы	Где используется
Mindbox	<code>Mindbox\MobileClient</code>	InitDevice, InitClient, RemoveDevice, Code, CheckCode, EditUser, IsUserExist, PushClick	<code>MindboxGrpcService</code> — мобильная авторизация и push
Mindbox	<code>Mindbox\UserClient</code>	Info, Orders, SendOSMICard	<code>MindboxUserService</code> — программа лояльности
Feedbacks	<code>Feedbacks\MobileClient</code>	App, Store, Order, Categories, ReasonsByOrder, ReasonsByStore, CanBeSaved	<code>FeedbackGRPCService</code>
Feedbacks	<code>Feedbacks\PortalFeedbackServiceClient</code>	Delete, List, Validate	Админка отзывов
Orders	<code>Orders\OfflineClient</code>	ByClient, GetById	<code>OrderGRPCService</code> — офлайн-заказы

Направление только исходящее — монолит выступает клиентом микросервисов, gRPC-сервер он не поднимает.

Особенность автозагрузки: в `composer.json` каталог `modules/grpc/` подключён как PSR-4 с пустым префиксом (`"": "modules/grpc/"`), из-за чего сгенерированные классы попадают в корень пространства имён.

Экспорт фидов

Фиды формируются только из консоли и сохраняются файлами. Полный список получателей — в [03-business-domains.md](#), раздел «Фиды».

Фронтенд

Каталог `frontend/`. Vue 2 + Webpack 5. Не SPA: каждой странице соответствует свой бандл, общая «обвязка» подключается отдельно.

Сборка

Файл	Роль
<code>frontend/webpack.config.js</code>	Основной конфиг
<code>frontend/webpack.helpers.js</code>	Поиск entry points и генерация алиасов
<code>frontend/package.json</code>	Скрипты и зависимости

Автоматический поиск entry points

Точки входа не перечисляются вручную. `getEntries()` в `webpack.helpers.js`:

1. обходит подкаталоги `src/modules/`;
2. в каждом модуле рекурсивно ищет файлы `_entry.js` внутри `pages/`;
3. формирует имя бандла из пути, заменяя разделители на дефис.

Примеры:

Файл	Имя бандла
<code>modules/catalog/pages/product/index/_entry.js</code>	<code>catalog-product-index</code>
<code>modules/catalog/pages/product/redesign/_entry.js</code>	<code>catalog-product-redesign</code>
<code>modules/cart/pages/redesign/_entry.js</code>	<code>cart-redesign</code>
<code>modules/base/pages/_entry.js</code>	<code>base</code> (бандл по умолчанию)

Всего в проекте **95 файлов** `_entry.js`. Новый модуль со страницей подхватывается сборкой без правок конфига.

Алиасы

Генерируются автоматически из каталогов первого уровня в `src/` (`getAliases`):

Алиас	Путь
~	frontend/src/
Modules	frontend/src/modules/
Layouts	frontend/src/layouts/
Helpers	frontend/src/helpers/
Blocks	frontend/src/blocks/
Assets	frontend/src/assets/
Vendor	frontend/src/vendor/

`jquery` подключён как `external` и берётся из глобальной переменной `jQuery`.

Вывод сборки

Режим	Каталог
<code>dev (FRONTEND_APP_ENV не равен prod)</code>	<code>www/build/dev/</code>
<code>prod</code>	<code>www/build/<хэш>/</code>

Структура: `<имя бандла>/scripts.js` и `<имя бандла>/styles.css`, общие чанки — `chunks/vendor.js` и `chunks/layout-*.js`. Аксеты (`fonts/`, `icons/`, `images/`) копируются из `src/assets/`. Манифест — `www/build/manifest.json`.

Каталог сборки полностью очищается перед каждым запуском.

Скрипты

Скрипт	Команда	Когда
<code>build-dev</code>	<code>NODE_ENV=development webpack</code>	Разовая сборка для разработки
<code>build-prod</code>	<code>NODE_ENV=production webpack</code>	Прод-сборка
<code>watch</code>	<code>NODE_ENV=development webpack --watch</code>	Разработка
<code>eslint-git</code>	линтер по diff	Перед коммитом

Важно: `NODE_ENV`, который выставляют прт-скрипты, не влияет на режим самого `webpack`. `webpack.config.js` определяет `mode` (минификация, source maps) исключительно по переменной `FRONTEND_APP_ENV` из `.env`: `mode = FRONTEND_APP_ENV === 'prod' ? 'production' : 'development'`. Если `.env` не содержит `FRONTEND_APP_ENV=prod`, сборка через `build-prod` всё равно соберётся в `dev`-режиме несмотря на `NODE_ENV=production`.

Только переменные с префиксом `FRONTEND_*` пробрасываются в бандл через `DefinePlugin`.

Глобальные SCSS

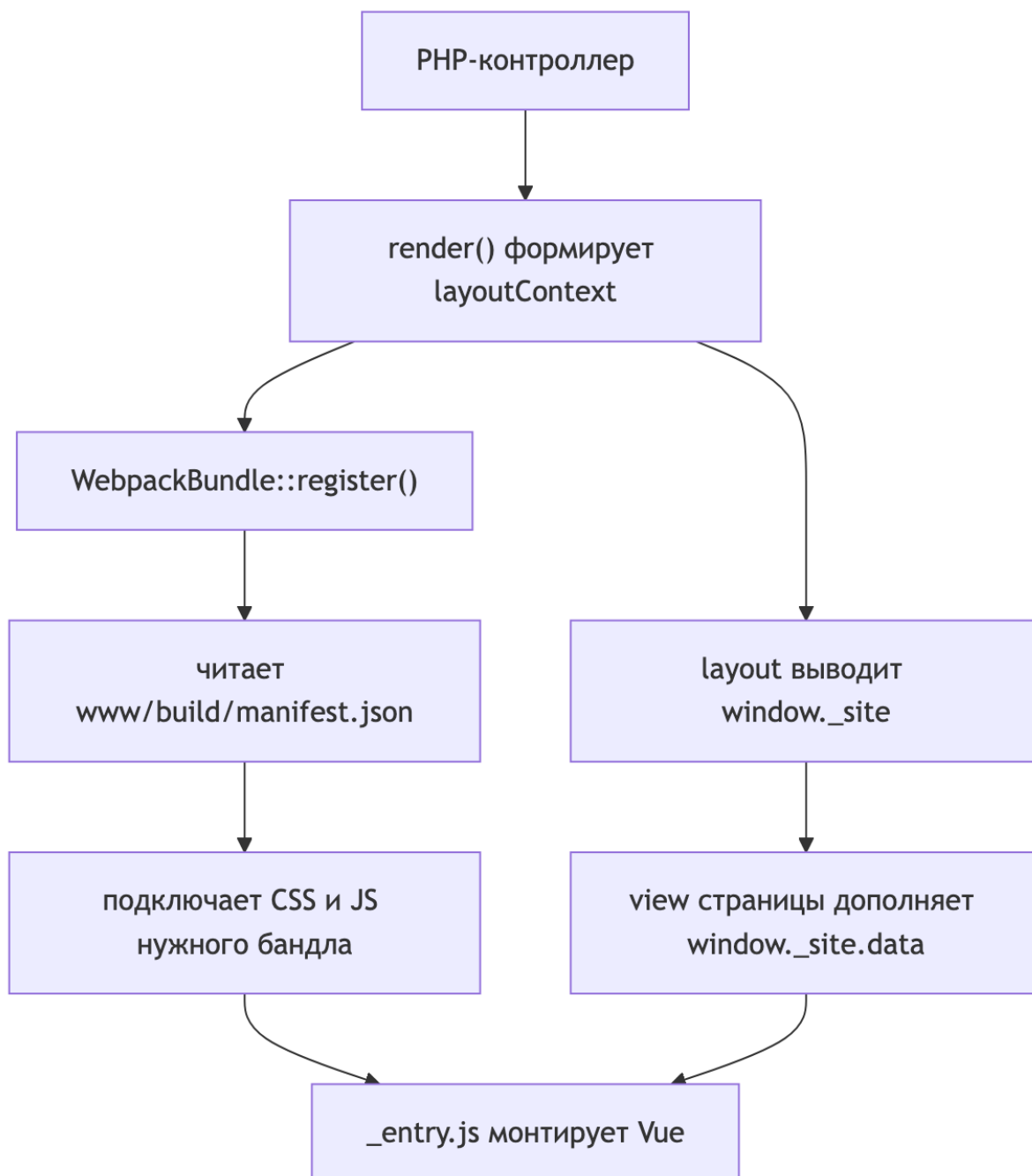
В каждый `.scss` файл через `sass-loader.additionalData` автоматически подставляются импорты:

```
Helpers/scss/deprecated/colors  
Helpers/scss/deprecated/rem  
Helpers/scss/deprecated/typography  
Helpers/scss/mixins  
Helpers/scss/deprecated/media-queries  
Helpers/scss/deprecated/blocks
```

Переменные и миксины доступны без явного импорта. Обратная сторона — эти файлы нельзя удалить без правки сборки, см. [09-pitfalls.md](#).

Связка PHP и Vue

Порядок загрузки



Выбор бандла

`themes/basic/assets/WebpackBundle.php` сопоставляет `controllerId/actionId` с именем бандла (например, `product/index` → `catalog/category`). Контроллер может задать бандл напрямую через свойство `entryName`. Если бандла нет в манифесте, подключается `base` — только обвязка.

`modules/cms/controllers/FrontendController.php` при рендере формирует `layoutContext` и определяет URL сборки.

Контракт `window._site`

Данные с сервера передаются одним глобальным объектом. Формируется в `modules/layout/service/BaseContextService.php`:

```
[
  'data'      => [...], // данные страницы и глобального UI
  'locales'   => [...], // строки локализации
  'routing'   => [...], // маршруты
  'settings'  => [...], // версия сборки, Sentry, Flomni и прочее
  'experiments' => ['all' => [...], 'current' => [...]],
]
```

Объект выводится в layout (`themes/basic/views/layouts/base.php`), а конкретная страница дополняет его:

```
Object.assign(window._site.data, <?= json_encode($context) ?>);
Object.assign(window._site.settings, <?= json_encode($settings ?? []) ?>);
Object.assign(window._site.locales, <?= json_encode($locales) ?>);
```

На клиенте доступ идёт через `frontend/src/layouts/base/workspace/context.js`, который разворачивает точечные ключи в объекты и предоставляет методы:

Метод	Что возвращает
<code>getData(key)</code>	Данные страницы
<code>getLocale(key)</code>	Строку локализации
<code>getRoute(key)</code>	Маршрут
<code>getSetting(key)</code>	Настройку
<code>getExperiment(alias)</code>	Признак участия в эксперименте
<code>getExperimentVariant(alias)</code>	Вариант эксперимента

В компонентах часть этих методов подмешивается миксином `frontend/src/layouts/base/shared/mixins/context.js` — но не один в один: `getData` там называется `getContext()`, а `getExperiment` / `getExperimentVariant` в этот миксин не входят (экспериментами компоненты пользуются через прямой импорт `context` или отдельный миксин). Миксин также добавляет вычисляемые свойства `context`, `locales`, `routing`, `settings` и метод `getSidebar()`.

Схемы и типизации у этого контракта нет. Переименование ключа локали не ловится на сборке и проявляется только в рантайме.

Способы получения данных

Способ	Где применяется
SSR через <code>_site.data</code>	Товар, категория, профиль, блоки главной
Дополнение в view страницы	Карточка товара, контекст новой корзины
Легаси-глобальные переменные	Старая корзина: <code>window.cartData</code> , <code>window.isGuest</code>
Запросы к API в рантайме	Сервисы в <code>layouts/base/workspace/</code> через <code>axiosService</code>

Обязка страницы

`frontend/src/layouts/base/init.js` подключается почти во всех entry points и делает:

- инициализацию Sentry и глобальных стилей;
- монтирование хедера, футера, таб-бара, опросов, уведомлений (несколько корневых Vue-инстансов);
- подключение аналитики: `dataLayer`, `Mindbox`, `Diginetica`, `ecommerce`-слушатели;
- автозагрузку **всех** файлов из `src/blocks/**` через `import.meta.webpackContext` ;
- наполнение Vuex данными экспериментов и гео.

Для минимальных страниц есть облегчённый вариант — `modules/empty/pages/_entry.js` .

Модули

58 модулей в `frontend/src/modules/` . Крупнейшие:

Модуль	Файлов	Есть страницы	Назначение
<code>ui-kit</code>	301	нет	Дизайн-система
<code>catalog</code>	229	да	Карточка товара, категория, поиск, фильтры
<code>cart</code>	131	да	Корзина и чекаут (старый и новый)
<code>orders</code>	50	да	Успешный заказ, оплата, возвраты
<code>landing</code>	42	да	Маркетинговые лендинги
<code>profile</code>	41	да	Профиль, заказы, бонусы, сертификаты
<code>base</code>	37	да	Общие хелперы, страница ошибки, бандл по умолчанию
<code>material</code>	33	да	Страницы о материалах

Модули без страниц — библиотеки сервисов, подключаемые из обвязки: `captcha` , `diginetica` , `marketing` , `smart-script` , `fitting` , `poll` , `video` , `dresscode` , `long-read` , `rates` , `subscription` , `content` , `ui-kit` .

Партнёрские модули с отдельными лендингами: `dreamers` , `oskelly` , `cosmoscow` , `garderobo` .

ui-kit

`frontend/src/modules/ui-kit/` , 301 файл.

Раздел	Содержимое
<code>controls/</code>	<code>Button</code> , <code>ButtonV2</code> , <code>Checkbox</code> , <code>Radio</code> , <code>Toggle</code> , <code>Tabs</code> , <code>Slider</code> , <code>Chip</code> , <code>Counter</code> , <code>Dots</code> , <code>Bullets</code> , <code>Close</code> , <code>IconButton</code> , <code>QuantityController</code>
<code>typography/</code>	<code>BodyText</code> , <code>BodyExtraText</code> , <code>HeadlineText</code> , <code>TitleText</code> , <code>LabelText</code> , <code>DisplayText</code> , <code>StrikeThroughText</code>
<code>forms/</code>	<code>TextField</code> , <code>TextAreaField</code> , <code>SearchField</code> , <code>ComplexField</code> , <code>FieldWrapper</code> , <code>RatingInput</code> , директива маски
<code>popups/</code>	<code>AlertPopup</code> , <code>ConfirmPopup</code> , <code>PopupTemplate</code> , <code>PopupManager</code>
<code>navigation/</code>	<code>Breadcrumbs</code>
<code>icons/</code>	Более 100 SVG-компонентов; часть разложена по размерам (<code>18/</code> , <code>20/</code> , <code>24/</code> , <code>32/</code> , <code>40/</code> , <code>48/</code> , <code>64/</code>), часть — по смысловым каталогам (<code>account/</code> , <code>arrows/</code> , <code>payment-method/</code> , <code>social/</code> и др.), часть лежит прямо в корне <code>icons/</code>
<code>dropdown/</code>	<code>Dropdown</code> , <code>DropDownList</code> и части
<code>elements/</code>	<code>AppBar</code> , <code>EmptyState</code> , <code>Rating</code> , <code>Countdown</code>
<code>fixed/</code>	<code>FixedElement</code> , <code>Tooltip</code> и менеджеры
<code>media/</code>	<code>VideoPlayer</code>
<code>lists/</code> , <code>loaders/</code> , <code>qr/</code> , <code>debug/</code>	Вспомогательные
<code>shared/</code>	<code>PropsHelper</code> , константы, SCSS-переменные

Паттерн единого источника пропсов

Пропсы компонента описываются один раз в `<Компонент>/shared/props.js` . Каждый проп — объект Vue-описания с опциональным ключом `storybook` .

```
// КОМПОНЕНТ
import PropsHelper from 'Modules/ui-kit/shared/helpers/PropsHelper';
import * as props from './shared/props';

export default {
  props: PropsHelper.getVueProps(props), // ключ storybook вырезается
};
```

Для историй используется `PropsHelper.getStorybookArgs(props)`. Файлов `shared/props.js` в `ui-kit` — 15.

Storybook

На ветке `master` **Storybook не настроен**: каталога `frontend/.storybook/` нет, в `package.json` нет соответствующих скриптов, хотя `frontend/README.md` их упоминает. Конфигурация и истории существуют в отдельной ветке разработки.

При этом в коде есть 20 файлов `.stories.js` и демо-страницы модуля `storybook` (`storybook/propup`, `storybook/tooltip`) со своими бандлами.

Состояние

Глобальный Vuex

`frontend/src/layouts/base/workspace/store.js`:

Модуль стора	Источник
<code>cart</code>	<code>Modules/cart/store/cart.js</code>
<code>checkout</code>	<code>Modules/cart/store/checkout.js</code>
<code>catalog</code>	<code>Layouts/base/store/catalog.js</code>
<code>product</code>	<code>Modules/catalog/stores/product.js</code>
<code>poll</code>	<code>Layouts/base/store/poll.js</code>
<code>site</code>	<code>Layouts/base/store/site.js</code> — брейкпоинты, пол, эксперименты, признак десктопа
<code>user</code>	<code>Layouts/base/store/user.js</code> — авторизация, гео
<code>userNotifications</code>	<code>Layouts/base/store/userNotifications.js</code>

Инициализация Vue — `frontend/src/layouts/base/workspace/vue.js`.

Страница может регистрировать свой модуль динамически: главная добавляет `page` через `store.registerModule('page', pageStore)`.

Легаси-стор старой корзины

`frontend/src/modules/cart/ancient/store.js` — отдельный Vuex-стор, читающий `window.cartData`. Используется только старой корзиной. Новый чекаут работает с глобальным стором.

Синглтоны вне Vuex

Все в `frontend/src/layouts/base/workspace/`: `axios.js`, `axiosService.js`, `popupManager.js`, `tooltipManager.js`, `dropdownManager.js`, `scrollBlocker.js`, `storage.js`, `globalLoader.js`, `layoutEventManager.js`, `orderManager.js`, `notificationsManager.js`, а также доменные сервисы `catalogService.js`, `cartService.js`, `userService.js`.

Аналитика и внешние сервисы

Подключаемые из PHP

Сервис	Где подключается
Google Tag Manager, dataLayer	<code>themes/basic/views/layouts/parts/marketing/head.php</code>
Diginetica	там же
Яндекс.Метрика	там же
VK Retargeting	там же
Mindbox tracker	<code>themes/basic/views/layouts/parts/marketing/body_end.php</code>
Flomni (чат)	там же, настройки из <code>_site.settings</code>
Roistat	там же

Обёртки в JS

Файл в workspace/	Обёртка над
<code>dataLayer.js</code>	<code>Modules/marketing/services/dataLayer/DataLayer</code>
<code>ecommerce.js</code>	<code>Modules/marketing/services/ecommerce/Ecommerce</code>
<code>gtag.js</code>	<code>Modules/marketing/services/gtag/Gtag</code>
<code>yandexMetrika.js</code>	<code>Modules/marketing/services/yandexMetrika/YandexMetrika</code>
<code>mindbox.js</code>	<code>Modules/marketing/services/mindbox/MindboxService</code>
<code>digineticaSearch.js</code> , <code>digineticaTracking.js</code> , <code>digineticaAnyQuery.js</code>	Diginetica
<code>garderoboService.js</code>	<code>Modules/garderobo/services/GarderoboService</code>
<code>smartScriptService.js</code>	AppsFlyer Smart Script
<code>flomniService.js</code> , <code>criteoService.js</code> , <code>gdeSlon.js</code> , <code>vkRetargetingService.js</code>	Соответствующие сервисы

AdvCake — исключение: подключается не через PHP-layout и не через общую обвязку, а напрямую в двух местах на клиенте — `modules/cart/ancient/marketing.js` (легаси-корзина) и `modules/orders/pages/success/_entry.js` (страница успешного заказа), через глобальную функцию `window.advcake_order()`.

Страницы дополнительно задают тип страницы: `dataLayer.setPageType()` ,
`yandexMetrika.setPageType()` .

Капча

Модуль `frontend/src/modules/captcha/` , синглтоны `workspace/captcha/yandexCaptcha.js` и `googleCaptcha.js` , компонент `HybridCaptcha.vue` . Ключ Яндекс-капчи — из переменных `FRONTEND_YANDEX_SMART_CAPTCHA_*` .

Параллельный редизайн

Часть интерфейса существует в двух версиях одновременно.

Карточка товара

Версия	Бандл	Корневой компонент
Старая	catalog-product-index	pages/product/index/ProductPage.vue
Новая	catalog-product-redesign	pages/product/redesign/ProductPage.vue

Выбор делает `modules/product/controllers/ProductController.php` :

- query-параметр `?redesign=12stz` — принудительно новая версия;
- A/B-тест `NOVADEV-5523` , вариант `NOVADEV-5523_1` — новая версия;
- параллельно работает эксперимент `NOVADEV-6049` .

Внутри новой версии разделение на десктоп и мобильную идёт по `state.site.isDesktop` .

Корзина

Версия	Бандл	Точка монтирования	Стор
Старая	cart	#vue-app	cart/ancient/store + window.cartData
Новая	cart-redesign	#cart-content	Глобальный стор

Выбор в `CartController::actionIndex()` : для админов и при `?redesign=12stz` .

Категория

Активна `CategoryPageNew.vue` . Старая `CategoryPage.vue` осталась в репозитории, но нигде не импортируется.

Прочие признаки

Двойные компоненты `SizesPopup` и `SizesPopupNew` , варианты слайдера `SMALL_REDESIGN` и `BIG_REDESIGN` , CSS-классы `new-design` , `catalog-button__redesign` , `login--redesign` , флаг `redesign: true` в блоках информации о товаре.

Легаси

Область	Что это
<code>blocks/</code>	44 SCSS и 9 JS на jQuery — довыюшная вёрстка и поведение, грузится на каждой странице
<code>vendor/scripts/</code>	Сторонние библиотеки вне npm: jQuery и плагины, slick, masonry, featherlight, intlTelInput
<code>helpers/scss/deprecated/</code>	Устаревшие переменные и миксины, подставляются автоматически во все SCSS
<code>layouts/base/ancient/</code>	Старая система попапов (<code>window.basePopup</code>) рядом с <code>PopupManager</code> из ui-kit
<code>modules/cart/ancient/</code>	API, валидация и маркетинг старой корзины

Дублирующиеся компоненты:

Область	Дубли
Карточка товара	<code>components/Product/</code> , <code>components/ProductRedesign/</code> , <code>ProductContentMobile/</code>
Краткая информация	<code>Product/desktop/ProductShortInfo.vue</code> и <code>ProductShortInfo/ProductShortInfo.vue</code>
Попап размеров	<code>popups/SizesPopup/</code> и <code>popups/SizesPopupNew/</code>
Кнопки	<code>controls/Button/</code> и <code>controls/ButtonV2/</code>
Корзина	старый стек и <code>components/redesign/</code>
Категория	<code>CategoryPage.vue</code> (мёртвый) и <code>CategoryPageNew.vue</code>

Структура `frontend/src/`

```
frontend/src/
├── assets/      # шрифты, иконки, изображения (копируются в сборку)
├── blocks/      # легаси jQuery + SCSS, подключаются глобально
├── helpers/     # DOMHelper, SCSS-миксины, устаревшие глобальные стили
├── layouts/
│   ├── base/   # основная обвязка: init.js, workspace/, store/, components/
│   └── empty/   # минимальная обвязка
├── modules/     # 58 функциональных модулей
├── stories/     # заготовки Storybook
└── vendor/     # вендорные скрипты и стили
```

Инфраструктура

CI/CD

GitLab CI собирает образы через Kaniko, публикует в `hub.12stz.dev` и деплоит Helm-чартом в Kubernetes. Конфиг — `.gitlab-ci.yml`.

Стадии и условия запуска

Стадия	Джобы	Когда
<code>build backend deps</code>	<code>build backend deps</code>	MR в develop/master или пуш в них, только если менялся <code>composer.*</code>
<code>build frontend</code>	<code>build frontend</code> , <code>build frontend prod</code>	При изменениях в <code>frontend/**</code> ; прод-сборка по тегу <code>v*.*.*</code>
<code>build images</code>	<code>build images</code> , <code>build images prod</code>	Матрица: <code>nginx</code> и <code>backend</code>
<code>stat analyse</code>	<code>phpstan</code> , <code>phpstan_src</code> , <code>codesniffer</code> , <code>eslint</code>	На MR
<code>tests</code>	<code>tests</code> , <code>phpunit</code> , <code>openapi_coverage</code> , <code>ui-test</code> , <code>api-test</code>	На MR; UI и API тесты вручную
<code>deploy</code>	<code>deploy:dev fix mobiledev sandbox stg prod</code>	Вручную
<code>rollback</code>	<code>rollback:*</code>	Вручную, требует <code>MIGRATION_COUNT</code>

Глобальные переменные: `PROJECT_NAME=web-monolith`, `DOCKERHUB_URL=hub.12stz.dev`, `HELM_NAMESPACE=web-$ENVIRONMENT`, `PHP_IMAGE_TAG=v1.0.13`, `COVERAGE_MIN_PERCENTAGE=5`, `OPENAPI_COVERAGE_MIN_PERCENTAGE=50`.

Собираемые образы

Джоба	Образ	Особенности
<code>build backend deps</code>	<code>.../backend-deps:\$ {DEPS_COMMIT}</code>	Composer install с SSH-ключом из Vault
<code>build frontend</code>	<code>.../frontend:\$ {FRONTEND_IMAGE_TAG}</code>	<code>.env</code> берётся из Vault, тег — хэш последнего коммита в <code>frontend/</code>
<code>build images</code>	<code>.../nginx:\$ {CI_COMMIT_SHA}</code> , <code>.../backend:\$ {CI_COMMIT_SHA}</code>	Матрица компонентов
<code>build images prod</code>	То же с <code>\$ {CI_COMMIT_TAG}</code>	Раннер <code>k2-prod</code>

Базовые образы: `php8.2-monolith:v1.0.13`, `nginx:1.25.4-alpine`, `node:18.17.1-alpine`.
Отдельно собирается `postfix:v1.0.0`.

Dockerfile-ы — в `.deploy/docker/`. Dockerfile-frontend кроме `npm run build-prod` также получает `.env` из Vault (`vault kv get ... > ./env`) прямо на этапе сборки образа, устанавливает зависимости (`npm install`) и удаляет `.env` после сборки (`rm ./env`) — то есть секреты фронтенда на короткое время оказываются в build-контексте образа.

Контроль качества

Проверка	Порог или правило
<code>phpstan</code>	<code>phpstan.neon</code> , уровень 2, весь проект
<code>phpstan_src</code>	<code>phpstan_6lvl_src.neon</code> , уровень 6, при изменениях в <code>src/**</code>
<code>codesniffer</code>	<code>phpcs --standard=./phpcs.xml</code>
<code>eslint</code>	Читает лимит из <code>frontend/eslint/config/max-warnings.txt</code> (660) в переменную <code>MAX_WARNINGS</code> , но фактическая команда — <code>npx eslint . --quiet --format json ...</code> — флаг <code>--max-warnings</code> не передаётся, а <code>--quiet</code> отбрасывает warning-уровень целиком, оставляя только ошибки; лимит де-факто не применяется
<code>phpunit</code>	Покрытие строк не ниже 5%
<code>openapi_coverage</code>	Покрытие спецификациями не ниже 50%
<code>tests</code>	<code>check-migrate-trigger.sh</code> — парные миграции таблиц истории

Окружения

Джоба	Окружение	Тег образа	URL
<code>deploy:dev</code>	dev	<code>CI_COMMIT_SHA</code>	dev.12stz.tech
<code>deploy:fix</code>	fix	<code>CI_COMMIT_SHA</code>	fix.12stz.tech
<code>deploy:mobiledev</code>	mobiledev	<code>CI_COMMIT_SHA</code>	mobiledev.12stz.tech
<code>deploy:sandbox</code>	sandbox	<code>CI_COMMIT_SHA</code>	sandbox.12stz.tech
<code>deploy:stg</code>	stg	<code>CI_COMMIT_SHA</code>	staging.12stz.tech
<code>deploy:prod</code>	prod	<code>CI_COMMIT_TAG</code>	12storeez.com

Namespace — `web-{окружение}`. Команда деплоя:

```
helm upgrade web-monolith .deploy/helm --install --atomic --timeout 600s \
  -f .deploy/helm/${ENVIRONMENT}.values.yaml \
  --set nuc.defaultImageTag=${IMG_TAG} \
  --set nuc.configMaps.web-monolith-envs.data.BUILD_VERSION=${CI_COMMIT_REF_NAME}
```

Миграции при деплое

Helm-хуки основного чарта:

Хук	Команда
pre-upgrade	<code>yii migrate --interactive=0</code>
pre-upgrade	<code>yii migrate --migrationPath=@yii/rbac/migrations/</code>
pre-upgrade	<code>yii migrate --migrationPath=@yii/log/migrations/</code>
post-upgrade	<code>yii message config/i18n.php</code>

Откат

Джобы `rollback:*` используют отдельный чарт `.deploy/migrations/` и релиз `web-monolith-rollback:`

```
helm upgrade web-monolith-rollback .deploy/migrations \
  --set nuc.migrationCount=${MIGRATION_COUNT} \
  --set nuc.defaultImageTag=${IMG_TAG}
```

Джоба выполняет `php ./yii migrate/down {{ $.Values.global.migrationCount }} --interactive=0`. **Количество миграций задаётся вручную** переменной `MIGRATION_COUNT`, без неё джоба падает ещё до `helm upgrade` (явная проверка `if [[ -z "${MIGRATION_COUNT}" ]]; then exit 1; fi`).

Похоже на баг: чарт `.deploy/migrations` — обёртка над сабчартом `nxs-universal-chart` (алиас `nuc`, см. `Chart.yaml`), и шаблон читает значение из `global.migrationCount` — то есть из `nuc.global.migrationCount` с точки зрения родительского чарта. CI же передаёт `--set nuc.migrationCount=...` — на один уровень выше нужного. Значение по умолчанию `global.migrationCount: 1` в `.deploy/migrations/values.yaml`, судя по всему, никогда не переопределяется этим `--set`, и `migrate/down` всегда откатывает ровно 1 миграцию независимо от `MIGRATION_COUNT`. Стоит перепроверить перед следующим использованием отката.

Топология в Kubernetes

Веб-поды

Деплоймент `web-monolith` — три контейнера в одном поде:

Контейнер	Роль
<code>web-monolith-backend</code>	php-fpm, слушает <code>127.0.0.1:9000</code> , <code>pm.max_children=360</code>
<code>web-monolith-nginx</code>	Проксирующее на fpm, JSON-логи доступа, <code>/metrics</code> только с разрешённых IP
<code>web-monolith-postfix</code>	Локальный SMTP на <code>127.0.0.1:25</code> и OpenDKIM

Реплики: прод и stg — 3, остальные окружения — 1.

Дополнительно:

- `web-monolith-admin` — тот же набор контейнеров с конфигурацией под админку (`memory_limit=5G` в базовом `values.yaml`, но **50G** в `prod.values.yaml`), 1 реплика; для сравнения, у обычного `web-monolith-backend` — `memory_limit=3G` везде;
- `web-monolith-error-pages` — nginx-заглушка для 5xx.

Общее хранилище: PVC `pvc-web-monolith-sfs` (ReadWriteMany, 10 Гб), монтируется в `www/images`, `www/uploads`, `www/export`.

Вход: Istio VirtualService и nginx Ingress → сервис `web-monolith-http:80`. Health-check — `/api2/healthcheck/ping`.

Воркеры и кроны

Всё описано в `.deploy/helm/values.yaml` с переопределениями в `.deploy/helm/{env}.values.yaml`:

Тип	Количество
Всего worker-деплойментов (без <code>web-monolith</code> , <code>web-monolith-admin</code> , <code>web-monolith-error-pages</code>)	68
Из них — воркеры MySQL-очереди (<code>yii queue/listen</code>)	25
Из них — консьюмеры RabbitMQ, ESB-консоли и прочие обработчики	43
CronJob-ов	109

Это важно: реальное асинхронное поведение прода задаётся Helm-values, а не кодом приложения. Консольная команда может существовать без воркера, и наоборот.

Основные воркеры MySQL-очереди:

Деплоймент	Очередь	Реплик в проде
web-monolith-queue	default	1
web-monolith-queue-analytics	analytics	3
web-monolith-queue-crm	crm	1
web-monolith-queue-crm-customer	crmcustomer	1
web-monolith-queue-crm-discount	crmdiscount	1
web-monolith-queue-crm-payment	crmpayment	1
web-monolith-queue-mindbox	mindbox	1
web-monolith-queue-receipts	receipts	1
web-monolith-queue-reindexproduct	reindexproduct	1
web-monolith-queue-reindexproductstocks	reindexproductstocks	1
web-monolith-queue-esbstocks	esbstocks	1
web-monolith-queue-giftcardsunlock	giftcardsunlock	1
web-monolith-queue-orderscounter	orderscounter	1

Консьюмеры RabbitMQ для обмена заказами (по 3 реплики в проде):

Деплоймент	Команда
<code>web-monolith-queue-onec-outbox-rabbit-create</code>	<code>rabbit-outbox-transactions/create onec</code>
<code>web-monolith-queue-onec-outbox-rabbit-update</code>	<code>rabbit-outbox-transactions/update onec</code>
<code>web-monolith-queue-rcrm-outbox-rabbit-create</code>	<code>rabbit-outbox-transactions/create rcrm</code>
<code>web-monolith-queue-rcrm-outbox-rabbit-update</code>	<code>rabbit-outbox-transactions/update rcrm</code>
<code>web-monolith-queue-mindbox-outbox-rabbit-create</code>	<code>rabbit-outbox-transactions/create mindbox</code>
<code>web-monolith-queue-mindbox-outbox-rabbit-update</code>	<code>rabbit-outbox-transactions/update mindbox</code>
<code>web-monolith-queue-onec-inbox-rabbit-update</code>	<code>rabbit-inbox-transactions/update onec</code>
<code>web-monolith-queue-rcrm-inbox-rabbit-create</code>	<code>rabbit-inbox-transactions/create rcrm</code>
<code>web-monolith-queue-rcrm-inbox-rabbit-update</code>	<code>rabbit-inbox-transactions/update rcrm</code>

Консьюмеры платёжного outbox:

Деплоймент	Команда
<code>web-monolith-queue-dolyame-outbox-rabbit-request</code>	<code>rabbit-bnpl-outbox-transactions/request dolyame</code>
<code>web-monolith-queue-podeli-outbox-rabbit-request</code>	<code>... request podeli</code>
<code>web-monolith-queue-yandex-split-outbox-rabbit-request</code>	<code>... request split</code>
<code>web-monolith-queue-yandex-pay-outbox-rabbit-request</code>	<code>... request pay</code>
<code>web-monolith-queue-yandex-sbp-outbox-rabbit-request</code>	<code>... request yandex-sbp</code>

Прочие консьюмеры: обработка изображений (`imageresize` , `image_s3_upload`), заказы Lamoda, импорт VIC из RCRM, привязка отзывов, остатки магазинов, сообщения в Mattermost, импорт пользователей, обмен номенклатурой, офлайн-заказы, отправка чеков в микросервис.

Группы CronJob-ов: экспорт фидов, обработка outbox и inbox, отправка платёжных уведомлений (каждую минуту), переиндексация товаров, очистка (истёкшие JWT, старые SMS-логи, outbox), загрузка WSDL 1C.

Файл `config/cron-web` описывает расписания времён bare-metal и в Kubernetes не применяется.

Асинхронная обработка

MySQL-очередь

Параметр	Значение
Компонент	<code>components/SqlQueue.php</code>
Таблица	<code>default_queue</code> — имя жёстко захардкожено в <code>SqlQueue</code> , <code>TABLE_PREFIX</code> не применяется
Колонки	<code>queue</code> , <code>run_at</code> , <code>payload</code> , <code>ukey</code>
Имена очередей	<code>src/Queue/SqlQueueName.php</code> — 22 константы
Воркер	<code>php ./yii queue/listen <имя></code>

Идемпотентность обеспечивается уникальным `ukey`: при попытке вставить дубль ловится `IntegrityException` с кодом 1062, и задание тихо пропускается. Задание удаляется при извлечении — семантика «не более одного раза».

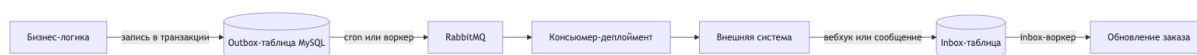
RabbitMQ

Параметр	Значение
Компонент	<code>components/rabbitmq/RabbitMQ.php</code> (php-amqplib)
Настройки	дurable-очереди, QoS prefetch = 1
Переменные	<code>RABBIT_HOST</code> , <code>RABBIT_PORT</code> , <code>RABBIT_USER</code> , <code>RABBIT_PASSWORD</code> , <code>RABBIT_VHOST</code>
Имена очередей	<code>src/Queue/RabbitMqQueueName.php</code> — 45 констант (40 уникальных строковых значений — часть констант ссылается на уже объявленные)

Публикация номенклатуры идёт в exchange `nomenclature` с routing key `nomenclature.delta.product.site`.

Outbox и Inbox

Обмен с внешними системами построен на транзакционном outbox.



Порядок работы:

1. бизнес-логика в той же транзакции пишет запись в outbox-таблицу;
2. cron (`process-*outbox-cron`) или воркер читает таблицу и публикует JSON в Rabbit через `OrderIntegrationOutboxFacade` / `OutboxTransactionService`;
3. консьюмер выполняет обработчик: SOAP-вызов 1C, REST RetailCRM, Mindbox, API платёжного провайдера;
4. при ошибке публикации запись остаётся, и её подхватит следующий проход крона.

Повторные попытки и идемпотентность:

- счётчик попыток на транзакцию хранится в Redis в группе `Integration` с TTL из `PaymentSystemDictionaryEnum::OUTBOX_CACHE_KEY_TTL` ;
- максимум попыток берётся из параметров CMS (`outbox*_max_retries`) либо из значений по умолчанию;
- при превышении лимита сообщение отклоняется без возврата в очередь;
- обработчики различают исключения: `OutboxTransactionRetryException` — не удалять запись из очереди (`ack()` для повтора), `OutboxTransactionSkipException` — удалить без логирования, `OutboxTransactionErrorException` — удалить и записать ошибку через `LogExtendedMessage` (отдельного канала уведомлений на этот случай нет, только структурированный лог).

Входящее направление устроено зеркально: `InboxTransactionStrategy` плюс воркеры `rabbit-inbox-transactions/{create,update}` {сервис} и cron как резервный путь.

Кэширование

Кластеры Redis

Кластер	Переменные	Используется
Основной	<code>REDIS_CLUSTER_NODES</code> либо <code>REDIS_HOST / REDIS_PORT / REDIS_PASSWORD / REDIS_DATABASE</code>	<code>RedisInterface</code> , компоненты <code>cache</code> , <code>cacheMain</code> , <code>cacheYiiT</code> , <code>denylist JWT</code> , счётчики <code>outbox</code>
Нового чекаута	<code>REDIS_NEW_CLUSTER_NODES</code>	<code>NewRedisInterface</code> , компонент <code>NewCache</code>

Если переменная с узлами кластера заполнена — используется кластерное подключение, иначе одиночное.

Формирование ключа кэша

В `config/web.php` префикс ключа собирается из литерала и четырёх переменных составляющих (итого 5 частей, склеенных без разделителя):

Составляющая	Как определяется
Литерал <code>old_cache</code>	Константа
Номер сервера	По имени хоста
Английская версия	Хост содержит <code>en.</code>
Признак авторизации	Наличие cookie <code>_identity</code>
Мобильное устройство	Результат <code>Mobile_Detect->isMobile()</code>

Итоговые префиксы: `cacheMain` — `p` + префикс, `cacheYiiT` (переводы `i18n`) — `t` + префикс.

Следствие: один логический ключ существует во множестве вариантов, у гостя и авторизованного пользователя кэши разные. Подробнее — [09-pitfalls.md](#).

Переключение кластера по URL

В `config/web.php` компонент `cache` подменяется на `NewCache` (кластер нового чекаута) по функции `isNewCheckout()`, которая проверяет **URI запроса** — без обращения к каким-либо флагам-параметрам. `NewCheckoutCacheConfigurator` (`src/Modules/Cart/Service/`) — отдельный класс с единственным методом `isCacheEnabled(): bool`, читающим флаг `NEW_CHECKOUT_CACHE_ENABLED`; он не участвует в подмене `Yii::$app->cache`, а служит gate-проверкой для отдельного Redis-кэша чекаута в `CheckoutCacheService`. Это два независимых механизма, которые легко перепутать.

Кэш схемы БД

Компонент `cacheSchema` — `FileCache` в `@runtime/cache/dbSchema`, TTL 1 час. Redis для схемы не используется. Регистрируется только в `config/web.php` — у «чистой» консоли (без веб-расширения конфига) этого компонента нет.

Инвалидация

Массовый сброс через консоль отключён — `CacheController` возвращает подсказку использовать `RedisInterface`. Точечная инвалидация: `TagDependency::invalidate()` (отзывы, лукбуки), `cacheYiiT->flush()` (после правки переводов в админке), `geoCache->flush()`, `Cache::flush()` через Redis.

Наблюдаемость

Sentry

Переменная	Назначение
<code>SENTRY_ENABLED</code>	Включение
<code>SENTRY_DSN</code>	Адрес проекта
<code>SENTRY_SAMPLE_RATE</code>	Доля отправляемых ошибок
<code>SENTRY_TRACES_SAMPLE_RATE</code> , <code>SENTRY_ENABLE_TRACING</code>	Трейсинг
<code>SENTRY_PROFILES_SAMPLE_RATE</code> , <code>SENTRY_ENABLE_PROFILER</code>	Профилирование
<code>SENTRY_DEBUG</code>	Отладка

Инициализация — в `config/common.php`. Сервис — `src/Monitoring/Sentry/Service/SentryService.php`. Не отправляются: `NotFoundHttpException`, `TooManyRequestsHttpException`, `RequestValidatorException`. Тег релиза берётся из `BUILD_VERSION`, который задаётся при деплое.

Метрики

`/metrics` → `metrics/metrics/index` (`MetricsController`), формат Prometheus text 0.0.4, библиотека `prometheus/client_php`. Доступ ограничен на уровне nginx списком IP. Сервис `web-monolith-metrics` отдаёт порты 8080 (`nginx stub_status`) и 9000.

Логи

Настройка — `config/log.php`.

Цель	Класс	Куда	Уровни
<code>errors</code>	<code>DbTargetExtended</code>	БД <code>log_db</code>	<code>error</code> , <code>warning</code>
<code>default</code>	<code>DbTargetExtended</code>	БД <code>log_db</code>	<code>info</code> , <code>trace</code> , <code>profile</code>
<code>stderr</code>	<code>LogJsonTarget</code>	<code>php://stderr</code>	<code>error</code> , <code>warning</code> (категория <code>GRAFANA_ERR</code>)
<code>stdout</code>	<code>LogJsonTarget</code>	<code>php://stdout</code>	<code>info</code> (категория <code>GRAFANA</code>)
<code>container</code>	<code>FileTarget</code>	<code>@runtime/logs/container.log</code>	Вызовы DI при <code>LOG_CONTAINER_CALLS</code>

Логи фронтенда — не цель логирования, а таблица `frontend_logs` через модель `FrontendLog`.

Обработка ошибок

Веб: `app\coreExtends\errorHandler\ExtendedErrorHandler`, действие `cms/frontend/error`. При недоступности апстрима nginx отдаёт заглушку из `web-monolith-error-pages`.

Конфигурация и секреты

Vault в CI

Этап	Что берётся
<code>build backend deps</code>	SSH-ключ <code>dev_ops/cicd/git_bot_ssh</code> для приватных Composer-репозиториев
<code>build frontend</code>	<code>.env</code> из <code>develop/backend/web/12storeez/frontend/{dev prod}/envs</code>
Деплой и откат	OIDC-токен GitLab → <code>kube-login</code>

Vault в Kubernetes

Синхронизация через `nuc-vault-secret-operator`:

Секрет	Путь в Vault	Содержимое
<code>web-monolith-secret-envs</code>	<code>backend/web/12storeez/{env}/envs</code>	Все чувствительные переменные; при изменении перезапускает деплоймент
<code>web-monolith-s3-envs</code>	<code>.../s3-envs</code>	Доступы к S3
<code>web-monolith-dkim-key</code>	<code>.../dkim-key</code>	Ключ DKIM для Postfix
<code>web-monolith-registry-hub-auth</code>	<code>common/registry_hub_auth</code>	Доступ к registry
TLS	<code>dev_ops/certificates/{домен}</code>	Сертификаты

Порядок применения конфигурации

1. Секрет Vault → переменные окружения во всех подах.
2. ConfigMap `web-monolith-envs` → несекретные переменные (`APP_ENV`, `BUILD_VERSION`).
3. `config/config.php` читает переменные в массив `$params`.
4. `config/common.php` и `config/web.php` конфигурируют компоненты из `$params`.
5. Helm-values задают тюнинг nginx, php-fpm, postfix, число реплик и расписания кранов.

Локально используется файл `.env` (в git не хранится, пример — `.env.example`). Для phpstan и phpcs есть `config/params_example.php`, для тестов — `.env.test`.

Переменные, которые есть в `config/config.php`, но отсутствуют в `.env.example`:

`REDIS_NEW_CLUSTER_NODES`, `BUILD_VERSION`.

Внешние интеграции

Сводная таблица

Система	Назначение	Протокол	Направление	Где код
Mindbox	CRM, лояльность, бонусы, рассылки, рекомендации	REST + gRPC	Двустороннее	modules/mindbox/ , src/Modules/Mindbox/ , modules/grpc/Mindbox/
RetailCRM	Заказы, клиенты, скидки, VIC-импорт	REST	Двустороннее	modules/retailCrm/ , src/Modules/Rcrm/
1C	Номенклатура, остатки, заказы, сертификаты	SOAP + вебхуки	Двустороннее	src/Modules/OneC/ (каталога modules/onec/ не существует)
Lamoda	Маркетплейс-заказы	REST	Двустороннее	modules/api2/controllers/OrdersController.php (actionCreateFromLamoda), modules/orders/services/LamodaOrderOutboxService.php , src/Modules/Order/Services/LamodaOrderService.php (каталога src/Modules/Lamoda/ не существует)
Mercaux	Офлайн-заказы в магазинах	REST	Входящее	modules/api2/ , OrderMethodEnum::MERCAUX
YooKassa	Оплата картой	REST	Двустороннее	modules/yandex/
Yandex Pay	Оплата	REST через биллинг-MC	Двустороннее	src/Integration/Payment/YandexPay/
Yandex Split	Рассрочка	REST через биллинг-MC	Двустороннее	src/Integration/Payment/YandexSplit/
Yandex SBP	Оплата по СБП	REST через биллинг-MC	Двустороннее	src/Integration/Payment/YandexSbp/
Tinkoff Dolyame	Оплата долями	REST	Двустороннее	modules/payment/components/Dolyame.php
Alpha Podeli	Оплата долями	REST	Двустороннее	src/Modules/Payment/Components/Podeli.php
ATOL	Фискализация чеков	REST	Исходящее	src/Modules/Receipt/ , modules/atol/
LifePay	Фискализация (легаси)	REST	Исходящее	modules/lifepay/components/Lifepay.php (компонент), modules/orders/jobs/LifepayJob.php , консоль lifepay
Boxberry	ПВЗ и доставка	REST	Двустороннее	modules/delivery/ , консоль boxberry
Accord Post	ПВЗ и доставка	REST	Двустороннее	modules/delivery/ , консоль accord
DaData		REST	Исходящее	components/Dadata.php

Система	Назначение	Протокол	Направление	Где код
	Подсказки адресов, ФИАС			
Яндекс.Геокодер	Геокодирование (несмотря на название класса)	REST	Исходящее	<code>components/Geocode.php</code> (APIURL — <code>geocode-maps.yandex.ru</code> , а не Google)
Google Geocode	Резервное/ альтернативное геокодирование	REST	Исходящее	<code>components/Google.php</code> (<code>maps.googleapis.com/maps/api/geocode/json</code>)
IpGeoBase	Гео по IP	Файловый парсинг	Входящее	консоль <code>ipgeobase-parse</code>
Diginetica	Поиск, подсказки, трекинг	REST + JS	Двустороннее	<code>modules/diginetica/</code> , <code>frontend/src/modules/diginetica/</code>
Elasticsearch	Поиск по каталогу	HTTP	Двустороннее	<code>src/Search/Elastic/</code>
Flomni	Онлайн-чат	JS-виджет	Клиентское	<code>themes/basic/views/layouts/parts/marketing/</code>
Garderobo	Подбор образов	REST + JS	Двустороннее	<code>src/Modules/Feed/Service/GarderoboFeedService.php</code> (сервер), <code>frontend/src/modules/garderobo/</code> + <code>frontend/src/layouts/base/workspace/garderoboService.js</code> (клиент)
Dresscode	Стилистические подборки (фид)	REST	Исходящее	<code>src/Modules/Feed/Service/DresscodeFeedService.php</code> , <code>Extractors/Xml/DresscodeFeedExtractor.php</code> (сервер; отдельного <code>src/Modules/Dresscode/</code> не существует), <code>frontend/src/modules/dresscode/services/DresscodeService.js</code> (клиент)
AppsFlyer	Мобильная атрибуция	JS Smart Script	Клиентское	<code>frontend/src/layouts/base/workspace/smartScriptService.js</code> , <code>frontend/src/modules/smart-script/services/SmartScriptService.js</code>
Admitad, GdeSlon	Партнёрские программы	REST + фиды	Исходящее	<code>components/Admitad.php</code> , <code>GdeSlon.php</code>
Criteo, VK, Google, Яндекс	Реклама и ретаргетинг	Фиды + JS	Исходящее	<code>modules/feed/extractors/</code>
PopMechanic	Сбор контактов	REST	Входящее	<code>PopmechanicController</code>
Mattermost	Оповещения команды	Через Notifications MS (не прямой webhook)	Исходящее	<code>src/Modules/Notifications/Console/NotificationsController.php</code> (<code>notifications-console/send-mattermost</code>) → <code>NotificationsClient->sendV2()</code>

Система	Назначение	Протокол	Направление	Где код
S3	Хранение изображений	S3 API	Двустороннее	<code>src/Common/S3/Configurator.php</code> , загрузка — <code>src/Modules/Product/Console/ImageS3ExportController.php</code> , очередь <code>RabbitMqQueueName::IMAGE_S3_UPLOAD</code>
Sentry	Мониторинг ошибок	HTTPS	Исходящее	<code>src/Monitoring/Sentry/</code>
Prometheus	Метрики	Pull / <code>metrics</code>	Входящее	<code>src/Modules/Metrics/</code>
SMS-коды авторизации	Отправка через Mindbox, не отдельный SMS-провайдер	gRPC	Исходящее	<code>src/Modules/Auth/Client/MindboxClient.php</code> → <code>MindboxGrpcService::sendCode()</code> → <code>MobileClient->Code()</code>
reCAPTCHA / Яндекс SmartCaptcha	Защита форм	REST + JS	Двустороннее	<code>src/Integration/Captcha/</code> , <code>frontend/src/modules/captcha/</code>
Huntflow	HR, вакансии	REST	Исходящее	<code>components/Huntflow.php</code> , <code>config/common.php</code>
5Post / ApiShip	Доставка (провайдер)	REST	Двустороннее	<code>DELIVERY_PROVIDER=apiship_fivepost</code> в <code>.env.example</code> , <code>src/Modules/Delivery/Dictionary/ProviderNameDictionary.php::FIVEPOST</code>
MaxMind	Гео по IP (наравне с IpGeoBase)	Локальная БД GeoIP	—	<code>src/Modules/Geo/MaxMind/Service/MaxMindService.php</code> , используется в <code>src/Modules/Geo/Common/Service/GeoService.php</code>
Notifications MS	Отправка уведомлений (в т.ч. в Mattermost)	REST/HTTP	Исходящее	<code>src/Modules/Notifications/Client/NotificationsClient.php</code>
Marshrut, SDT	Доставка (легаси-провайдеры)	REST	Двустороннее	<code>modules/delivery/components/</code> , регистрация в <code>config/common.php</code>
Giftcards MS	Подарочные сертификаты	REST	Двустороннее	<code>modules/giftcards/</code> , <code>BFB_MS_GIFTCARDS_HOST</code>
Yandex Tracker	Внутренний трекер задач (используется в мобильном профиле)	REST	Исходящее	<code>src/Modules/YandexTracker/</code> , <code>src/Modules/Mobile/Controllers/ProfileController.php</code>
PlatformCraft	Внешний счётчик/сервис	—	—	<code>components/PlatformCraft.php</code> , регистрация в <code>config/web.php</code> и <code>config/console.php</code>

Внутренние микросервисы

Монолит обращается к ним как клиент. Не все связи — gRPC: часть клиентов ходит по REST через общий хост `BFB_MS_HOST`, а не по отдельным gRPC-каналам, как можно было бы предположить по аналогии с Mindbox/Feedbacks/Orders.

Микросервис	Реальный клиент в DI	Протокол	Назначение
Yandex Pay / Split / SBP биллинг	<code>Application\Integration\Payment\Yandex*Client\BillingMsClient</code> (три конфигуратора, не единый <code>BillingMsClient::class</code> -синглтон)	REST	Платежи Yandex Pay / Split / SBP
Чеки	<code>Application\Modules\Receipt\Client\ReceiptClient</code> (хост <code>BFB_MS_HOST</code> , не отдельный <code>ReceiptMsClient</code>)	REST	Фискальные чеки
BFB MS	<code>MonolithBfbMsClient</code>	REST	Интервалы доставки
Отзывы	<code>Feedbacks\MobileClient</code> (зарегистрирован, но по коду не вызывается — используется только в DI); портал отзывов ходит через отдельный HTTP <code>FeedbackClient</code> (<code>FEEDBACK_BASE_URL</code>), а не через <code>PortalFeedbackServiceClient</code> (тот — неиспользуемый gRPC-стаб)	gRPC / HTTP	Отзывы
Orders MS	<code>Orders\OfflineClient</code>	gRPC	Офлайн-заказы
Mindbox MS	<code>Mindbox\MobileClient</code> , <code>UserClient</code>	gRPC	Мобильная авторизация, лояльность

Важная деталь: у gRPC-клиентов (`MobileClient`, `Feedbacks\MobileClient`, `OfflineClient`, `UserClient`) в DI нет отдельных переменных на каждый сервис — все они настроены через один общий `$params['GRPC_GEO_HOSTNAME']` (см. `di/ContainerRegister.php`), то есть в `.env.example` это выглядит как единая секция `#> Microservices → Geo (GRPC_GEO_*)`, а не набор `MINDBOX_MS_*` / `ORDERS_MS_*` / `FEEDBACKS_MS_*`, как можно было бы ожидать.

Подробнее о методах — [05-api.md](#), раздел «gRPC».

Особенности отдельных интеграций

Mindbox

Самая глубоко встроенная система. Используется одновременно как:

- источник истины по бонусам — локальные таблицы только отображают состояние;
- поставщик рекомендаций (`MindboxRecommendationsService`);
- канал мобильной авторизации через gRPC — фактически используются только `Code` и `CheckCode` (`MindboxGrpcService.php`); `InitDevice` вызывается лишь внутри сгенерированного клиента `modules/grpc/Mindbox/MobileClient.php`, из прикладного кода — не напрямую;
- получатель фида товаров (`MindboxFeedExtractor`);
- трекер поведения на клиенте (JS-обёртка `frontend/src/layouts/base/workspace/mindbox.js`).

Заказы уходят в Mindbox через outbox: таблицы `mindbox_create_transaction_outbox` и `mindbox_update_transaction_outbox`, консьюмеры `mindbox-outbox-rabbit-create/update`.

Следствие: недоступность Mindbox затрагивает и авторизацию в приложении, и отображение бонусов, и блок рекомендаций — то есть далеко не только рассылки.

1С

Обмен по SOAP, WSDL загружается кронем `cron-onec-settings/download-wsdl` (`CronOneCSettingsController::actionDownloadWsdL()`; отдельной команды `load-wsdl-cron` нет) и кэшируется. Входящие вебхуки — `/api/onec/webhooks/*` (баланс сертификата, статусы, офлайн-покупка, смена владельца). Исходящее направление — outbox-таблицы `onec_create_transaction_outbox` и `onec_update_transaction_outbox` плюс консьюмеры.

Настройки обмена хранятся в БД и обновляются командой `cron-onec-settings`.

Платёжные провайдеры

Все построены по одному шаблону: outbox-таблица, таблица уведомлений, обработчик транзакций, outbox-сервис, консольный обработчик уведомлений, cron и consumer в Helm. Детали — в [03-business-domains.md](https://business-domains.md), раздел «Платежи».

Современные Yandex-провайдеры (Pay, Split, SBP) ходят не напрямую, а через биллинговый микросервис. Dolyame и Podeli — напрямую в API провайдера.

Diginetica

Работает на двух уровнях: серверном (`DigineticaService` — поиск и подсказки) и клиентском (три JS-сервиса: поиск, трекинг, произвольные запросы). Отключение серверной части не отключает клиентский трекинг.

Переменные окружения

Полный список — в `.env.example` (корневой, для бэкенда) и `frontend/.env.example` (для фронтенда). Ниже — группировка по назначению **с реальными именами**; предыдущая версия документа во многих местах указывала предполагаемые по аналогии, а не фактические имена — таких переменных в `.env.example` нет.

Приложение и БД

```
APP_ENV, DEBUG_TOOLS, HOST, HOST_ADMIN, TABLE_PREFIX, BUILD_VERSION*
DSN, USERNAME, PASSWORD
LOG_DSN, LOG_USERNAME, LOG_PASSWORD
LOG_CONTAINER_CALLS
```

* — `BUILD_VERSION` читается в `config/config.php`, но в `.env.example` отсутствует (подставляется отдельно при деплое через Helm/ConfigMap). Переменных `APP_DEBUG`, `APP_URL` не существует.

Redis

```
REDIS_HOST, REDIS_PORT, REDIS_PASSWORD, REDIS_DATABASE
REDIS_CLUSTER_NODES
REDIS_NEW_CLUSTER_NODES*
NEW_CHECKOUT_CACHE_ENABLED
```

* — используется в `config/config.php`, но тоже отсутствует в `.env.example`.

Очереди и поиск

```
ELASTIC_HOST, ELASTIC_PORT, ELASTIC_USER, ELASTIC_PASSWORD, ELASTIC_ENABLED,
ELASTIC_FORCE_USE_INDEX
```

Единой переменной `ELASTIC_HOSTS` нет. Имя индекса каталога (`elastic_catalog_index`) — это не переменная окружения, а параметр CMS (`ElasticConfigurator::PARAMETER_INDEX_ALIAS`), задаётся миграцией и хранится в БД.

Аутентификация

```
JWT_PRIVATE_KEY, JWT_PUBLIC_KEY
MOBILE_API_KEY, MOBILE_DEVICE_ID_BAN_LIST
```

Платежи

```
ALPHA_PODELI_API_KEY           # Alpha Podeli (не PODELI_*)
YANDEX_SPLIT_MERCHANT_ID       # Yandex Split (нет отдельной группы YANDEX_SPLIT_*)
YANDEX_PAY_SBP_ONLY_MERCHANT_ID
ATOLV3_*, ATOLV4_*             # фискализация (не ATOL_*; в проде используется ATOLV4_*)
AGGREGATE_BILLING_MS_URL        # биллинговый микросервис (не BILLING_MS_*)
USE_RECEIPT_MS                  # флаг микросервиса чеков; сам клиент ходит на BFB_MS_HOST
```

Переменных `DOLYAME_*` в `.env.example` нет вовсе. Групп `YANDEX_PAY_*` / `YANDEX_SBP_*` как таковых тоже нет — только точечные ключи мерчантов.

CRM и интеграции

```
RCRM_API_ADDRESS, RCRM_API_KEY  # RetailCRM (не RETAIL_CRM_*)
API1C_URL, API1C_LOGIN, API1C_PASSWORD  # 1C (не ONEC_*)
FLOMNI_CLIENT_KEY
HUNTFLOW_*
```

Групп `LAMODA_*` (есть только `IT_ORDERS_LAMODA_WARNING_CHANNEL_ID` — канал Mattermost), `GARDEROBO_*`, `DRESSCODE_*` (в корневом `.env.example` нет; для Dresscode есть `FRONTEND_DRESSCODE_API_KEY` / `FRONTEND_DRESSCODE_COMPANY_ID`, но во `frontend/.env.example`), `MINDBOX_*` для gRPC-стабов не существует — gRPC-клиенты настроены через `GRPC_GEO_*`.

Доставка и гео

Boxberry, Accord Post, DaData и геокодеры настроены **захардкоженными ключами** в `config/common.php`, а не через переменные окружения — групп `BOXBERRY_*`, `ACCORD_*`, `DADATA_*`, `GOOGLE_GEOCODE_*` в `.env.example` нет.

```
BFB_MS_HOST, BFB_MS_GIFTCARDS_HOST
DELIVERY_PROVIDER # например, apiship_fivepost — для 5Post
```

Мониторинг

```
SENTRY_ENABLED, SENTRY_DSN, SENTRY_SAMPLE_RATE
SENTRY_TRACES_SAMPLE_RATE, SENTRY_ENABLE_TRACING
SENTRY_PROFILES_SAMPLE_RATE, SENTRY_ENABLE_PROFILER, SENTRY_DEBUG
```

S3 и почта

```
S3_KEY, S3_SECRET, S3_REGION, S3_ENDPOINT, S3_ENV_DIRECTORY
```

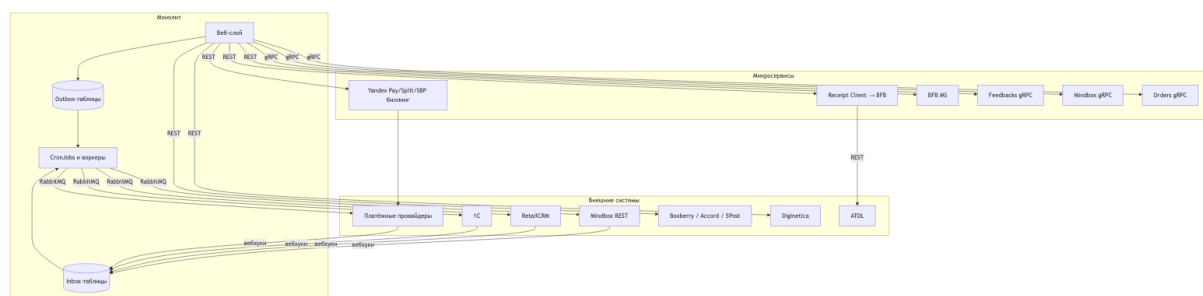
Переменной `S3_BUCKET` нет — имя бакета собирается иначе (через `S3_ENV_DIRECTORY` и конфигурацию клиента). Отдельных `SMTP_*` / `DKIM_*` тоже нет: почта настроена захардкоженным `Swift_SmtpTransport` (`host: 127.0.0.1`, `port: 25`) в `config/common.php`, из переменных — только `FROM_EMAIL`, `USE_FILE_TRANSPORT`; DKIM монтируется в под отдельным Vault-секретом (см. [07-infrastructure.md](#)).

Фронтенд

Только переменные с префиксом `FRONTEND_*` попадают в бандл (`frontend/.env.example`):

```
FRONTEND_APP_ENV
FRONTEND_SENTRY_DSN
FRONTEND_YANDEX_SMART_CAPTCHA_*
FRONTEND_GOOGLE_RECAPTCHA_API_KEY, FRONTEND_GOOGLE_RECAPTCHA_SCRIPT_URL # не
FRONTEND_GOOGLE_CAPTCHA_*
FRONTEND_DRESSCODE_API_KEY, FRONTEND_DRESSCODE_COMPANY_ID
```

Схема потоков данных



Неочевидные связи и подводные камни

Собранные при разборе кода места, где поведение системы расходится с ожиданиями. Каждый пункт проверен по коду и содержит ссылку на файлы.

Критичное

1. Приватный JWT-ключ лежит в репозитории

`jwt_keys/jwtRS256.key` и `jwtRS256.key.pub` **закоммичены в git**, при этом **ни одна строка кода на них не ссылается** — рантайм читает ключи из переменных окружения `JWT_PRIVATE_KEY` и `JWT_PUBLIC_KEY` (`src/Auth/Service/JwtService.php`).

Название файла (`RS256`) к тому же не соответствует используемому алгоритму — подпись идёт на `ES256`.

Что делать: убедиться, что ключи из репозитория нигде не использовались в проде, удалить их из истории. Каталог `.gitignore` уже содержит правило `/jwt_keys/*` — но сами файлы были закоммичены раньше и продолжают отслеживаться git (правило в `.gitignore` не ретроактивно), поэтому нужен ещё явный `git rm --cached .`

2. Два ActiveRecord-класса на таблицу `orders`

```
modules/orders/models/Order.php:467    → return '{{%orders}}';
src/Modules/Order/Model/Order.php:215  → return '{{%orders}}';
```

Оба класса живые и используются одновременно: легаси — в админке и на страницах оплаты, современный — в создании заказа и `outbox`.

Из этого следует:

- правило валидации, добавленное в один класс, **не применится** при сохранении через другой;
- хук `beforeSave` / `afterSave` одного класса не выполнится при записи через второй;
- поведение связей различается — легаси `getUser()` возвращает `ActiveQuery`, современный вызывает `→one()` и возвращает модель.

При правках логики заказа проверяйте оба класса.

3. Скрипт проверки парных миграций не работает при нескольких файлах

`check-migrate-trigger.sh` формирует список изменённых миграций и передаёт его в `grep` как одну строку:

```
files=$(git diff --name-only $from $target | grep 'migrations')
countOrders=$(grep -c '{{%orders}}' "$files")
```

Если в MR изменена **больше одной** миграции, "\$files" содержит перенос строки, и `grep` получает несуществующее имя файла. Дальнейшие сравнения работают на пустых значениях, проверка фактически не выполняется.

Отдельно: при отсутствии изменений в миграциях скрипт вызывает `return 0` вне функции — в `sh` это ошибка, а не успешный выход.

Правило при этом действующее и важное: **изменяя `{%orders}`, `{%order_positions}` или `{%products}`, добавляйте миграцию для соответствующей таблицы истории** — на автопроверку полагаться нельзя.

4. Дамп Redis закоммичен в репозиторий

`dump.rdb` закоммичен в репозиторий и не упомянут в `.gitignore` — при регенерации файла локально `git status` будет постоянно показывать его как изменённый. Файл небольшой (2.8 Кб), но это артефакт рантайма, которому не место в `git`. `.env` при этом корректно исключён через `.gitignore`.

Архитектурные ловушки

5. Зависимости между стеками инвертированы

Ожидание: `src/` — новый слой, `modules/` — легаси, зависимость идёт в одну сторону.

Реальность — зависимость двусторонняя:

- `src/` использует модели из `modules/`: `AbstractCartService` работает с `modules\cart\models\Cart`, `src\Modules\Order/` — с `modules\orders\models\OrderPosition`;
- `modules/` использует инфраструктуру из `src/`: Redis через `Application\Cache\Redis\Contract\RedisInterface`, очереди через `Application\Queue\`.

Практическое следствие: рефакторинг легаси-модели ломает современные сервисы, и наоборот. Оценивая радиус изменения, смотрите оба каталога.

6. Строгий `phpstan` применяется к коду, зависящему от слабо проверяемого

`phpstan_6lvl_src.neon` проверяет `src/` на уровне 6, а `phpstan.neon` проверяет остальной проект (`components`, `config`, `modules`, `www` и другие — **`src/` в его `paths` не входит**) на уровне 2, со 181 строкой `excludedPaths` (не `ignoreErrors` — секции с игнорируемыми паттернами ошибок в файле нет вовсе). Поскольку `src/` активно вызывает `modules/`, ошибки типизации на границе не отлавливаются: возвращаемые типы легаси-моделей на уровне 2 не проверяются.

7. Префикс `/mobile/v2/` обслуживают два модуля

Оба зарегистрированы в `config/modules.php`:

Модуль	Как маршрутизируется
<code>modules/mobilev2/</code> (id <code>mobilev2</code>)	Через общее правило <code><module>/<controller>/<action></code>
<code>src/Modules/MobileV2/</code> (id <code>mobile-v2</code>)	Через явные правила в <code>config/url.php</code>

Куда попадёт запрос, определяет **порядок правил** в `config/url.php` — срабатывает первое совпадение. Добавляя эндпоинт в `/mobile/v2/`, нужно проверить, не перехватывает ли его уже существующее правило другого модуля.

8. DI-контейнер — единая точка отказа

`di/ContainerRegister.php` — 1516 строк, **341** вызов `setSingleton()`, **381** импорт из обоих стеков, без разбиения на провайдеры по доменам. Дополнительная часть регистраций живёт в `config/common.php` (`container.singletons`), где, например, `AuthClientInterface` вне прода подменяется на `LocalClient`.

Ошибка в этом файле ломает всё приложение, а не отдельный модуль.

9. Обработчики платежей лежат в модуле заказов

События отмены платежей — `SendCancelToDolyameEvent`, `SendCancelToPodeliEvent`, `SendCancelToSplitEvent`, `SendCancelToYandexPayEvent`, `SendCancelToYandexSbpEvent` — находятся в `src/Modules/Order/AfterSaveEvent/`, а не в `src/Modules/Payment/`.

Всего в этом каталоге 26 обработчиков, запускаемых `EventRunner` после коммита транзакции заказа. Ища логику отмены платежа, начинайте с модуля `Order`.

10. Дублирующиеся сервисы с одинаковыми именами

Класс	Расположения
<code>PaymentTypeService</code>	<code>modules/delivery/service/</code> и <code>src/Modules/Delivery/Service/</code>
<code>Order</code>	<code>modules/orders/models/</code> и <code>src/Modules/Order/Model/</code>
<code>Parameter</code>	<code>modules/cms/models/</code> и <code>src/Modules/Cms/Model/</code> (обёртка)

Используются одновременно. При правке проверяйте, какой именно класс импортирован в месте вызова.

База данных

11. Имена таблиц не соответствуют именам сущностей

Ожидаемое	Фактическое
cart	shopping_carts
cart_items	shopping_cart_positions
order_items	order_positions
product_colors	не существует — products.color_id → dictionary_colors

Особое внимание: `order_positions.item_id` ссылается на `product_sizes.id`, а не на товар. Товар получается через размер. Поле с ожидаемым именем `product_size_id` в этой таблице отсутствует.

12. Товар в разных цветах — это разные записи каталога

Отдельной таблицы цветовых вариантов нет. Каждый цвет — своя строка в `products` со ссылкой `color_id`. Варианты одной модели связаны общими `group_guid` и `super_model_guid`.

Влияет на любой подсчёт «числа товаров», на фиды и на аналитику: одна модель в пяти цветах даёт пять записей.

13. Модели сертификатов игнорируют префикс таблиц

Все модели используют `{{%имя%}}`, чтобы подставлять `TABLE_PREFIX`. Модели подарочных сертификатов задают имена жёстко:

```
cms_giftcard_sell
cms_giftcard_use
cms_giftcard2user
```

При смене `TABLE_PREFIX` (например, в тестовом окружении) эти модели сломаются, остальные — нет.

14. Остаток товара живёт в трёх местах

`catalog_item_stock`, `product_stocks` и буферные таблицы (`catalog_item_quantity_buffer`, `catalog_item_stocks_quantity_buffer`, `product_size_stocks_temp`). Согласование не мгновенное — происходит по мере обработки очередей `esbstocks`, `reindexproductstocks` и кронов.

Проверяя «неправильный остаток», нужно смотреть все источники и состояние очередей, а не одну таблицу.

15. Sphinx присутствует, но не используется

В `composer.json` есть `yiisoft/yii2-sphinx`, в конфигах — `config/sphinx.php`. При этом компонент **не зарегистрирован** ни в `common.php`, ни в `web.php`, и ни один класс в `modules/` или `src/` его не импортирует. Единственное упоминание Sphinx во всём PHP-коде — сам файл `config/sphinx.php`.

Поиск работает исключительно на Elasticsearch (`src/Search/Elastic/`).

16. Кэш схемы БД — файловый, не Redis

Компонент `cacheSchema` — `FileCache` в `@runtime/cache/dbSchema` с TTL 1 час. Поскольку `@runtime` локален для пода, после миграции разные поды могут какое-то время видеть разные версии схемы. Сброс Redis на это никак не влияет.

Кэш

17. Ключ кэша зависит от пяти неявных факторов

В `config/web.php` префикс ключа для `cacheMain` и `cacheYiiT` формируется из литерала и четырёх переменных, склеенных без разделителя:

1. литерала `old_cache`;
2. номера сервера (по имени хоста);
3. признака английской версии (хост содержит `en.`);
4. признака авторизации (наличие cookie `_identity`);
5. признака мобильного устройства (`Mobile_Detect->isMobile()`).

Следствия:

- у гостя и авторизованного пользователя **разные кэши** одного и того же контента;
- у мобильного и десктопного User-Agent — тоже разные;
- сброс кэша «для страницы» затрагивает только одну комбинацию из многих;
- воспроизвести проблему с кэшем локально сложно — префикс зависит от хоста.

18. Компонент кэша подменяется в зависимости от URL — но не так, как кажется на первый взгляд

В `config/web.php` компонент `cache` подменяется на `NewCache` (другой кластер Redis, `REDIS_NEW_CLUSTER_NODES`) функцией `isNewCheckout()`, которая смотрит **только на URI запроса** — без чтения каких-либо флагов-параметров.

Легко перепутать это с похожим по названию классом `NewCheckoutCacheConfigurator` (`src/Modules/Cart/Service/`): у него единственный метод `isCacheEnabled(): bool`, читающий флаг `NEW_CHECKOUT_CACHE_ENABLED`, но он **не участвует** в подмене `Yii::$app->cache` — это gate-проверка для отдельного Redis-кэша чекаута внутри `CheckoutCacheService`. Два независимых механизма кэширования нового чекаута, у которых по имени можно было бы предположить связь, на деле не пересекаются.

Практическое следствие: один и тот же вызов `Yii::$app->cache` в одном запросе пойдёт в основной Redis, а в другом (по URI) — в кластер нового чекаута, независимо от значения `NEW_CHECKOUT_CACHE_ENABLED`. Отлаживая кэш нового чекаута, нужно проверять оба механизма по отдельности, не полагаясь на схожесть названий.

19. Массовый сброс кэша из консоли отключён

`CacheController` не сбрасывает кэш, а возвращает подсказку использовать `RedisInterface`. Точечная инвалидация — через `TagDependency::invalidate()`, `cacheYiiT->flush()` и группы Redis.

Группы, исключённые из массового сброса (`getFlushExcludedGroups()`): `Logs`, `ExchangeErrors`, `Auth`, `Integration`, `OrderCreateAction`, `WebhookOnec`, а также `RateLimiter` (`RateLimiterConfigurator::REDIS_GROUP`) — сбрасывать их нельзя, там лежат счётчики попыток `outbox`, состояние авторизации и лимиты запросов.

Фича-флаги

20. Часть флагов читается мимо `enum`

В `src/Modules/Common/Enum/Parameters.php` объявлено **47 констант**, а вызовов `Parameter::valueOf()` — **233 в 111 файлах**. Разница означает, что значительная часть параметров читается строковым алиасом напрямую.

Следствие: **список констант в `enum` не является полным перечнем фича-флагов системы**. Ища все места использования флага, нужно искать и по строковому алиасу, а не только по константе.

21. Один флаг часто существует в платформенных вариантах

Многие параметры продублированы с суффиксами `_WEB`, `_IOS`, `_ANDROID` (например, `IS_AVAILABLE_THANKYOU_RECOMMENDATIONS_*`). Включение функциональности «в целом» требует правки нескольких записей в таблице `parameters`.

Мобильные параметры дополнительно описаны в `src/Modules/Mobile/Enum/AndroidParametersEnum.php` и перечислениях `MobileV2`.

Фронтенд

22. Entry points находятся по соглашению, а не по списку

`getEntries()` в `frontend/webpack.helpers.js` обходит `src/modules/*/pages/**` и берёт каждый файл `_entry.js`. Имя бандла собирается из пути.

Следствия:

- переименование каталога страницы меняет имя бандла и **ломает связку с PHP** (`WebpackBundle.php` сопоставляет `controllerId/actionId` с именем бандла);

- если бандла нет в манифесте, страница получит `base` — только обвязку, без своей логики, и это не вызовет ошибки сборки.

23. `window._site` — контракт без схемы

Данные с PHP на клиент передаются одним глобальным объектом (`BaseContextService.php` → `layout` → `context.js`). Ни типизации, ни валидации нет.

Переименование ключа локали или данных не ловится ни сборкой, ни линтером — проявится только в рантайме, возможно, лишь на одной странице.

24. Устаревшие SCSS-файлы удалить нельзя

`sass-loader.additionalData` подставляет в **каждый** `.scss` шесть файлов, пять из которых лежат в `helpers/scss/deprecated/`:

```
deprecated/colors, deprecated/rem, deprecated/typography,
mixins, deprecated/media-queries, deprecated/blocks
```

Файлы помечены как устаревшие, но являются обязательной зависимостью сборки. Удаление любого из них ломает компиляцию всех стилей проекта.

25. Два стека UI живут параллельно

Область	Версии	Как выбирается
Карточка товара	<code>catalog-product-index</code> и <code>catalog-product-redesign</code>	<code>?redesign=12stz</code> или A/B-тест NOVADEV-5523
Корзина	<code>cart</code> и <code>cart-redesign</code>	<code>?redesign=12stz</code> или признак админа
Категория	<code>CategoryPage.vue</code> (мёртвый) и <code>CategoryPageNew.vue</code>	Новая всегда

Правка карточки товара или корзины требует решения, в какую версию она идёт — иначе изменение увидит только часть пользователей. `CategoryPage.vue` при этом мёртвый код: файл в репозитории есть, импортов нет.

26. Два независимых стора корзины

Старая корзина использует `frontend/src/modules/cart/ancient/store.js` и читает `window.cartData`, новый чекаут — глобальный `Vuex`. Общего состояния между ними нет.

27. Обвязка загружает все блоки на каждой странице

`layouts/base/init.js` через `import.meta.webpackContext` подключает **все** файлы из `src/blocks/**` (44 SCSS и 9 JS на jQuery) вне зависимости от того, нужны ли они странице. `jquery` при этом внешняя зависимость, берётся из глобального `jquery`.

28. Storybook на `master` не настроен

Каталога `frontend/.storybook/` нет, скриптов в `package.json` нет, хотя `frontend/README.md` их описывает. При этом в коде есть 20 файлов `.stories.js` и демо-страницы модуля `storybook` со своими бандлами. Конфигурация живёт в отдельной ветке разработки.

Инфраструктура

29. Рантайм-поведение прода описано в Helm, а не в коде

Что	Где	Количество
Консольные команды	<code>config/console.php</code>	94
Воркеры MySQL-очереди	<code>.deploy/helm/values.yaml</code>	25
Консьюмеры RabbitMQ и прочие обработчики очередей	<code>.deploy/helm/values.yaml</code>	43 (итого 68 worker-деплойментов)
CronJob-ы	<code>.deploy/helm/values.yaml</code>	109

Команда может существовать в коде без запущенного воркера — и тогда очередь копится молча. И наоборот: cron может вызывать команду, которой уже нет. Чтобы понять, что реально работает в проде, нужно смотреть Helm-values, а не только код.

30. Файл `config/cron-web` не используется

Описывает расписания времён bare-metal. В Kubernetes расписания задаются CronJob-ами из Helm-values. Файл вводит в заблуждение: правка в нём ни на что не влияет.

31. Три контейнера в одном поде, включая SMTP

Под `web-monolith` содержит `backend` (php-fpm), `nginx` и `postfix` с OpenDKIM. Почта отправляется на `127.0.0.1:25` внутри пода. Значит, при перезапуске пода письма в локальной очереди Postfix теряются, а масштабирование веб-слоя масштабирует и почтовый релей.

32. Откат миграций требует ручного ввода количества

Джобы `rollback:*` вызывают `helm upgrade web-monolith-rollback` с `--set nuc.migrationCount=${MIGRATION_COUNT}`, что по замыслу должно превращаться в `yii migrate/down {{ $.Values.global.migrationCount }}`.

Переменную задаёт человек. Без неё джоба падает; с неверным значением откатится не то количество миграций. Автоматической привязки «версия релиза → число миграций» нет.

Похоже на отдельный баг: чарт `.deploy/migrations` подключает `nxs-universal-chart` под алиасом `nuc` (`Chart.yaml`), а шаблон читает `global.migrationCount`, то есть с точки зрения родительского чарта — `nuc.global.migrationCount`. CI же передаёт `--set nuc.migrationCount=...` — на уровень выше нужного пути. Похоже, что откат всегда

использует дефолт `global.migrationCount: 1` из `.deploy/migrations/values.yaml`, а не значение `MIGRATION_COUNT` из пайплайна. Стоит проверить перед следующим использованием отката, а не полагаться на то, что CI и чарт согласованы.

33. Прод-образ фронтенда получает секреты Vault прямо в build-контекст

`Dockerfile-frontend` не ограничивается `npm run build-prod` — сначала он ставит `vault` в образ, забирает `.env` через `vault kv get ... > ./env`, потом `npm install` и только затем сборку; в конце `.env` удаляется (`rm ./env`). Тег образа считается по хэшу последнего коммита, затронувшего `frontend/`. Если изменение фронтенда не попало в этот каталог (например, правка PHP-шаблона в `themes/`), образ не пересоберётся.

34. `.env` для прод-сборки фронтенда приходит из Vault

Джоба `build frontend` забирает `.env` из Vault (`develop/backend/web/12storeez/frontend/{dev|prod}/envs`) и кладёт в контекст сборки. Локальный `.env.example` не отражает набор `FRONTEND_*`, реально используемый в проде. В бандл попадают только переменные с префиксом `FRONTEND_*`.

35. Переменные, отсутствующие в `.env.example`

`config/config.php` читает `REDIS_NEW_CLUSTER_NODES` и `BUILD_VERSION`, которых нет в `.env.example`. Первая включает кластер нового чекаута, вторая используется как тег релиза в Sentry и задаётся при деплое через `--set`.

36. Порог покрытия тестами — 5%

`COVERAGE_MIN_PERCENTAGE=5` в `.gitlab-ci.yml`. Порог покрытия OpenAPI выше — 50% (`OPENAPI_COVERAGE_MIN_PERCENTAGE`), но считается как отношение числа задокументированных операций к числу всех методов `action*` в контроллерах, то есть не проверяет соответствие спецификации фактическому поведению.

37. Сборка зависимостей запускается только при изменении `composer.*`

Джоба `build backend deps` имеет условие на изменение `composer.json` / `composer.lock`. В остальных случаях используется ранее собранный образ `backend-deps`. Это ускоряет пайплайн, но означает, что состояние `vendor/` определяется не текущим коммитом, а последним изменением зависимостей.

Идемпотентность и повторные попытки

38. MySQL-очередь работает по семантике «не более одного раза»

`components/SqlQueue.php` удаляет задание при извлечении. Если воркер упал в середине обработки, задание **потеряно** — повторной доставки нет.

Дедупликация обеспечивается уникальным `ukey` : при вставке дубля ловится `IntegrityException` с кодом 1062, и задание тихо пропускается. То есть повторная постановка того же задания не приведёт к двойной обработке, но и не гарантирует, что первая попытка завершилась.

39. Счётчики попыток outbox живут в Redis, а не в БД

Число попыток по транзакции хранится в Redis в группе `Integration` с TTL из `PaymentSystemDictionaryEnum::OUTBOX_CACHE_KEY_TTL` , лимит — из параметров CMS (`outbox*_max_retries`).

Следствие: сброс Redis обнуляет счётчики, и сообщения, уже отклонённые по лимиту, могут начать обрабатываться заново. Именно поэтому группа `Integration` исключена из массового сброса.

40. Классы исключений управляют судьбой сообщения

Обработчики outbox различают три ситуации:

Исключение	Поведение
<code>OutboxTransactionRetryException</code>	Не удалять сообщение из очереди, <code>ack()</code> — будет повтор
<code>OutboxTransactionSkipException</code>	Удалить из БД, <code>ack()</code> , без логирования ошибки
<code>OutboxTransactionErrorException</code>	Удалить из БД, <code>ack()</code> , залогировать через <code>LogExtendedMessage</code>

Отдельного канала «уведомления» для `ErrorException` нет — это структурированный лог, а не алерт. Выбросив в обработчике обычное исключение вместо одного из этих трёх, вы получите то же поведение, что и для `Throwable` по умолчанию (удаление + `ack()` + лог), но с потерей специфичной семантики `retry/skip`.

41. Порог `eslint` в CI фактически не применяется

Джоба `eslint` читает лимит предупреждений в переменную `MAX_WARNINGS` из `frontend/eslint/config/max-warnings.txt` (660), но реальная команда — `npx eslint . --quiet --format json ...` — не передаёт `--max-warnings` и вдобавок использует `--quiet` , который отбрасывает `warning`-уровень целиком. Порог существует только на бумаге; линтер в CI фактически репортит лишь ошибки.

42. У `web-monolith-admin` в проде `memory_limit` подсказывает до 50G

Базовый `values.yaml` задаёт `memory_limit=5G` для админского контейнера (против 3G у обычного `web-monolith-backend`), но `prod.values.yaml` переопределяет его до 50G. Такой разрыв обычно означает, что в админке когда-то ловили `Allowed memory size exhausted` на тяжёлой операции (массовый экспорт/импорт, отчёт) и подняли лимит с большим запасом, а не устранили причину потребления памяти. Стоит иметь в виду при отладке производительности конкретно прод-админки.

Быстрый чек-лист перед изменением

Меняете	Проверьте
Логiku заказа	Оба класса <code>Order</code> ; 26 обработчиков в <code>Order/AfterSaveEvent/</code>
Схему <code>orders / order_positions / products</code>	Миграция таблицы истории (автопроверка ненадёжна)
Поле в API	Все экстракторы: сайт, mobile v1, mobile v2, партнёры; обновить <code>docs/*/openapi.yml</code>
Эндпоинт <code>/mobile/v2/</code>	Порядок правил в <code>config/url.php</code> — два модуля на один префикс
Легаси-модель в <code>modules/</code>	Использование в <code>src/</code> — зависимость двусторонняя
Кэшируемые данные	Пять факторов префикса ключа; подмена <code>cache</code> на <code>NewCache</code> по URI (не по флагу)
Фича-флаг	Платформенные варианты <code>_WEB / _IOS / _ANDROID</code> ; поиск по строковому алиасу
Название страницы во фронтенде	Имя бандла и сопоставление в <code>WebpackBundle.php</code>
Карточку товара или корзину	В какую версию идёт правка — старую или редизайн
Новую консольную команду	Добавить воркер или CronJob в <code>.deploy/helm/values.yml</code>
Обработчик очереди	Класс выбрасываемого исключения определяет судьбу сообщения